HOUSEHOLD ELECTRICITY CONSUMPTION PREDICTION SYSTEM

A PROJECT REPORT

Submitted by

Kunal Mittal (24BCS12354)
Ansh Kumar (24BCS12097)
Vishal Kumar Shrivastava (24BCS12629)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

November 2026

BONAFIDE CERTIFICATE

Certified that this project report "Household Electricity Consumption Prediction System" is the bonafide work of "Kunal Mittal, Ansh Kumar and Vishal Kumar Shrivastava" who carried out the project work under my/our supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT **SUPERVISOR**

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

ABSTRACT 8
GRAPHICAL ABSTRACT 9
ABBREVIATIONS 10
SYMBOLS 11
CHAPTER 1. INTRODUCTION 12
1.1. Identification of Client / Need / Relevant Contemporary issue 12
1.2. Identification of Problem 12
1.3. Identification of Tasks 13
1.4. Timeline 13
1.5. Organization of the Report 14
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 15
2.1. Timeline of the reported problem 15
2.2. Existing solutions 15
2.3. Bibliometric analysis 16
2.4. Review Summary 16
2.5. Problem Definition 17
2.6. Goals/Objectives 17
CHAPTER 3. DESIGN FLOW/PROCESS 19
3.1. Evaluation & Selection of Specifications/Features 19
3.2. Design Constraints 21
3.3. Analysis of Features and finalization subject to constraints 21
3.4. Design Flow 22
3.5. Design selection 23
3.6. Implementation plan/methodology 23
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 27
4.1. Implementation of solution 27
CHAPTER 5. CONCLUSION AND FUTURE WORK 41
5.1. Conclusion 41
5.2. Future work 41
REFERENCES 43
APPENDIX 45

1. Plagiarism Report 45
2. Design Checklist 45
USER MANUAL 46

List of Figures

Figure 0.1*: Graphical abstract: end-to-end MLOps architecture of VoltCast* 9
Figure 1.1*: Gantt chart of the project timeline* 14
Figure 3.1*: Daily mean active power with 30-day rolling mean (strong yearly seasonality)* 19
Figure 3.2*: Average consumption by weekday and hour (morning and evening peaks)* 19
Figure 3.3*: Monthly pattern and share of energy per sub-meter* 20
Figure 3.4*: Box plots of hourly readings; points beyond the whiskers are IQR outliers* 20
Figure 3.5*: Alternative designs considered* 22
Figure 3.6*: Flowchart of the training and deployment algorithm* 24
Figure 3.7*: Detailed block diagram of the MLOps architecture* 24
Figure 4.1*: Correlation matrix of the measured variables* 27
Figure 4.2*: Validation MAE and R² of all models (best in orange, baselines in grey)* 28
Figure 4.3*: MLflow tracking UI: runs of every model, tuned variant and final model* 29
Figure 4.4*: Training vs validation MAE of each model (default hyper-parameters)* 30
Figure 4.5*: Validation curve of the decision tree: error against max_depth* 30
Figure 4.6*: Validation MAE before and after GridSearchCV tuning* 31
Figure 4.7*: Next-hour forecast vs actual consumption, last 14 days of the test set* 32
Figure 4.8*: Predicted vs actual scatter and residual distribution* 32
Figure 4.9*: Top 15 features by permutation importance* 33
Figure 4.10*: Peak-hour classifier on the test set: confusion matrix, ROC and precision-recall curves* 34
Figure 4.11*: VoltCast web application (React) with a live 24-hour forecast and peak-hour alerts* 35
Figure 4.12*: Prometheus target page: the VoltCast API is being scraped (state UP)* 37
Figure 4.13*: Grafana dashboard "VoltCast – Model Monitoring" fed by Prometheus* 37
Figure 4.14*: Population Stability Index of monitored features (train vs test)* 38
Figure 4.15*: Test error across sub-groups (dashed line: overall MAE)* 38
Figure 4.16*: Responsible-AI section of the web application* 39
Figure M.1*: Web application – live forecast* 47
Figure M.2*: Web application – model performance, over-fitting and tuning* 48
Figure M.3*: Web application – peak-hour classifier metrics* 49
Figure M.4*: Grafana monitoring dashboard* 49
Figure M.5*: Prometheus targets page* 50
Figure M.6*: Streamlit dashboard (Docker / local) – forecast tab* 50

List of Tables

Table 0.1*: Standards referred to in the project* 7
Table 0.2*: List of abbreviations* 10
Table 0.3*: List of symbols* 11
Table 1.1*: Identified tasks* 13
Table 2.1*: Timeline of the reported problem* 15
Table 2.2*: Bibliometric comparison of existing solutions* 16
Table 2.3*: Mapping of project objectives to course outcomes* 18
Table 3.1*: Selected feature specification (31 features)* 20
Table 3.2*: Design constraints* 21
Table 3.3*: Comparison of alternative designs* 23
Table 3.4*: Software modules of the implementation* 25
Table 4.1*: Training and validation performance of all models (default hyper-parameters)* 28
Table 4.2*: Hyper-parameter tuning results* 30
Table 4.3*: Held-out test performance (Jun–Nov 2010)* 31
Table 4.4*: Peak-hour classifiers on the validation set* 33
Table 4.5*: REST API endpoints* 34
Table 4.6*: Automated test suite results* 36
Table 4.7*: Achievement of objectives* 39
Table A.1*: Design checklist* 45

List of Standards

Table 0.1*: Standards referred to in the project*

Standard	Publishing Agency	About the standard	Where used
ISO/IEC 23053:2022	ISO / IEC	Framework for Artificial Intelligence systems using Machine Learning; defines the ML pipeline components (data acquisition, pre-processing, training, evaluation, deployment).	Ch. 3.2, 3.6
ISO/IEC 25010:2023	ISO / IEC	Systems and software quality model (functional suitability, reliability, maintainability, portability).	Ch. 3.2, 4.1
IEC 62056 (DLMS/COSEM)	IEC	Electricity metering data exchange; defines how smart meters expose load-profile readings such as those used as model inputs.	Ch. 3.1
IS 16444	Bureau of Indian Standards	A.C. static direct-connected smart energy meters; relevant for deploying the system on Indian household smart meters.	Ch. 3.2
ISO 50001:2018	ISO	Energy management systems: requirements for measuring, analysing and improving energy performance.	Ch. 1.1, 3.2
OpenMetrics (Prometheus exposition format)	CNCF	Text format for exposing service metrics; the API's /api/metrics endpoint follows it so Prometheus can scrape it.	Ch. 3.6, 4.1
Digital Personal Data Protection Act, 2023	Government of India	Rules for processing digital personal data; applies when real household meter data is used.	Ch. 3.2, 4.1

ABSTRACT

Residential buildings account for a large share of national electricity demand, yet most households receive only a monthly bill and have no forward-looking view of their consumption. Utilities, in turn, need short-term load forecasts at household level to plan demand response as smart meters are rolled out. This project develops **VoltCast**, a Household Electricity Consumption Prediction System that forecasts a household's hourly active power for the next 1–48 hours, flags likely peak-consumption hours, and is built end to end with Machine Learning Operations (MLOps) practice.

The UCI "Individual Household Electric Power Consumption" dataset (2,075,259 one-minute readings over almost four years) is cleaned of 1.252% missing values, checked for outliers, resampled to 34,589 hourly records and turned into 31 leakage-free features. Nine regression models covering the algorithms of the course (linear regression, decision tree, k-nearest neighbours, support vector machine, random forest) plus ridge regression, gradient boosting and two naive baselines are trained on a chronological split, tuned with GridSearchCV using time-series cross-validation, and tracked in MLflow. Histogram gradient boosting gives the best forecast: test MAE 0.293 kW, MSE 0.187, R^2 0.623, a 22.2% MAE reduction over the persistence baseline. A tuned decision-tree classifier detects peak hours with 80.3% accuracy, 77.9% recall and ROC-AUC 0.873. The experiments also show over-fitting clearly: the default decision tree reaches zero training error but the worst validation error, which tuning reduces from 0.506 to 0.380 kW.

The models are versioned with DVC and the MLflow model registry, and served by a Flask REST API with input validation. They run in Docker (a five-service Docker Compose stack, plus Kubernetes manifests) and on the Vercel cloud behind a React web application. A GitHub Actions pipeline reproduces the DVC pipeline, applies quality gates, runs 11 automated tests and builds the container on every change and weekly. Prometheus collects service metrics that Grafana displays, every prediction is logged, and a drift monitor based on the Population Stability Index correctly detected seasonal drift and recommended retraining. Fairness, transparency, privacy and security are analysed, including error across groups of hours.

Keywords: load forecasting, smart meter, gradient boosting, decision tree, GridSearchCV, MLOps, MLflow, DVC, Flask, Docker, Prometheus, Grafana, responsible AI.

GRAPHICAL ABSTRACT

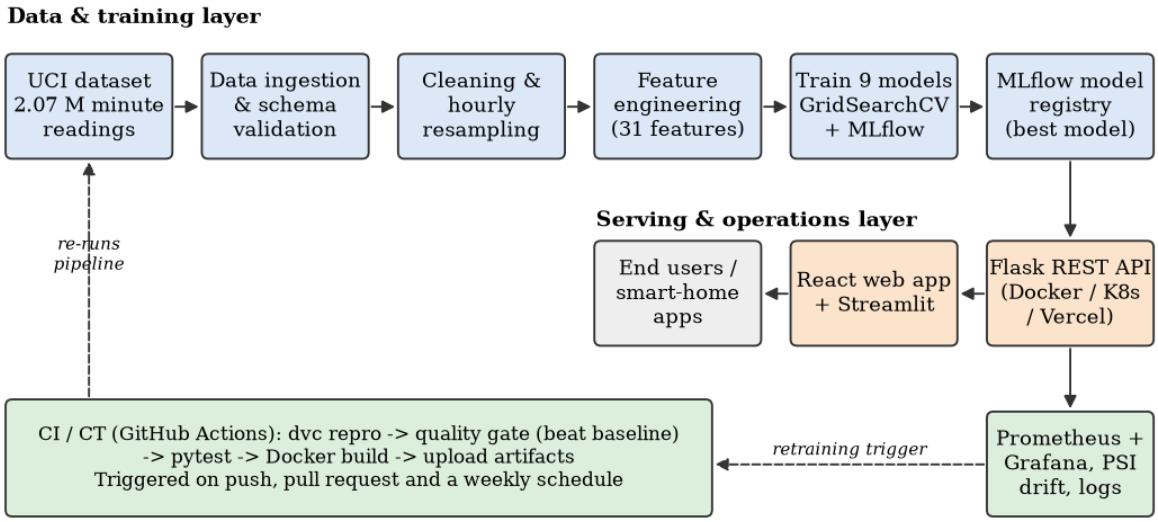


Figure 0.1*: Graphical abstract: end-to-end MLOps architecture of VoltCast*

ABBREVIATIONS

Table 0.2*: List of abbreviations*

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
AUC	Area Under the (ROC) Curve
CI/CD	Continuous Integration / Continuous Deployment
CO	Course Outcome
CT	Continuous Training
CV	Cross-Validation
CNN	Convolutional Neural Network
DVC	Data Version Control
EDA	Exploratory Data Analysis
FN / FP / TN / TP	False negative / false positive / true negative / true positive
HGB	Histogram-based Gradient Boosting
IEA	International Energy Agency
IQR	Inter-Quartile Range
K8s	Kubernetes
KNN	K-Nearest Neighbours
kW / kWh	Kilowatt / Kilowatt-hour
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MLOps	Machine Learning Operations
MSE	Mean Squared Error
NILM	Non-Intrusive Load Monitoring
PSI	Population Stability Index
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
R^2	Coefficient of determination
SVM / SVR	Support Vector Machine / Support Vector Regression
UCI	University of California, Irvine (ML Repository)

SYMBOLS

Table 0.3*: List of symbols*

Symbol	Meaning
y_t	Hourly mean global active power at hour t (kW)
$\hat{y}_t$	Predicted active power at hour t (kW)
lag_k	Value of the target k hours earlier, y_{t-k}
μ_w, σ_w	Rolling mean and standard deviation over the previous w hours
n	Number of evaluation samples
α	Ridge regularisation strength
η	Learning rate of gradient boosting
k	Number of neighbours in KNN
C, ε	SVR penalty and epsilon-insensitive margin
σ_r	Standard deviation of test residuals (used for prediction intervals)
h	Forecast horizon in hours
τ	Peak-hour threshold (75th percentile of training consumption, kW)
p_i, q_i	Share of reference and current data in bin i (PSI)

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary issue

Electricity use in buildings is one of the largest and fastest-growing components of energy demand. The International Energy Agency reports that buildings account for roughly 30% of global final energy consumption, and space cooling, water heating and appliances in homes are major drivers of evening peak demand [1]. In India, the domestic (household) sector consumes about one quarter of all electricity sold, and its share keeps rising with appliance ownership and air-conditioner adoption [2].

Three groups of stakeholders need short-term household consumption forecasts:

- **Households** want to know how much energy and money they will use tomorrow, so they can shift flexible loads (washing machines, water heaters) and avoid bill shock; time-of-day tariffs make this more valuable.
- **Distribution utilities** run demand-response and peak-shaving programmes and must forecast load at fine granularity. India's Revamped Distribution Sector Scheme targets the installation of about 250 million prepaid smart meters, which will generate exactly the kind of high-frequency data this project uses [3].
- **Smart-home and energy-management vendors** need a deployable, monitored prediction service rather than a one-off notebook model, which is the gap that MLOps practice addresses [4], [5].

Studies of production ML systems show that the model code is a small part of a real ML system and that most failures come from data dependencies, configuration and lack of monitoring [4], [23]. A consumption predictor that silently degrades when seasons or occupants change is a contemporary, documented issue.

1.2. Identification of Problem

Households and utilities lack a reliable, continuously maintained way to anticipate a single home's electricity demand over the next hours and to know in advance which hours will be peak hours. Household load is highly volatile, depends on human routines, weekday, season and appliance usage, and drifts over time. Meter readings contain gaps, noise and

extreme values. Models built once and left unattended become stale, and there is no feedback that tells the operator when this happens.

1.3. Identification of Tasks

The work was divided into the following tasks (Table 1.1), which also define the structure of this report:

Table 1.1*: Identified tasks*

#	Task	Output	Chapter
T1	Survey of the problem, existing forecasting methods and MLOps literature	Problem definition, objectives	1, 2
T2	Selection of a public, realistic smart-meter dataset and EDA	Data understanding, figures	3.1
T3	Design of features, constraints (incl. ethics) and alternative designs	Selected design	3
T4	Data pipeline: ingestion, cleaning, outliers, resampling, features, versioning with DVC	Versioned datasets	3.6, 4.1
T5	Model development: 9 algorithms, GridSearchCV tuning, over-fitting analysis, MLflow registry	Registered forecaster	4.1
T6	Peak-hour classifier with accuracy, precision, recall, F1, ROC	Peak alert model	4.1
T7	Deployment: Flask REST API, Docker, Kubernetes, Vercel cloud, React web app	Running services	4.1
T8	Automation and operations: CI/CD, tests, Prometheus, Grafana, logging, drift monitoring	CI pipeline, dashboards	4.1
T9	Responsible-AI analysis: fairness, transparency, privacy, security	Model card, bias analysis	3.2, 4.1
T10	Validation, documentation, report and user manual	This report	4, 5, Manual

1.4. Timeline

The project was executed over twelve weeks. Figure 1.1 shows the Gantt chart; tasks overlap where the pipeline allowed parallel work (for example, the web application was built while CI was configured).

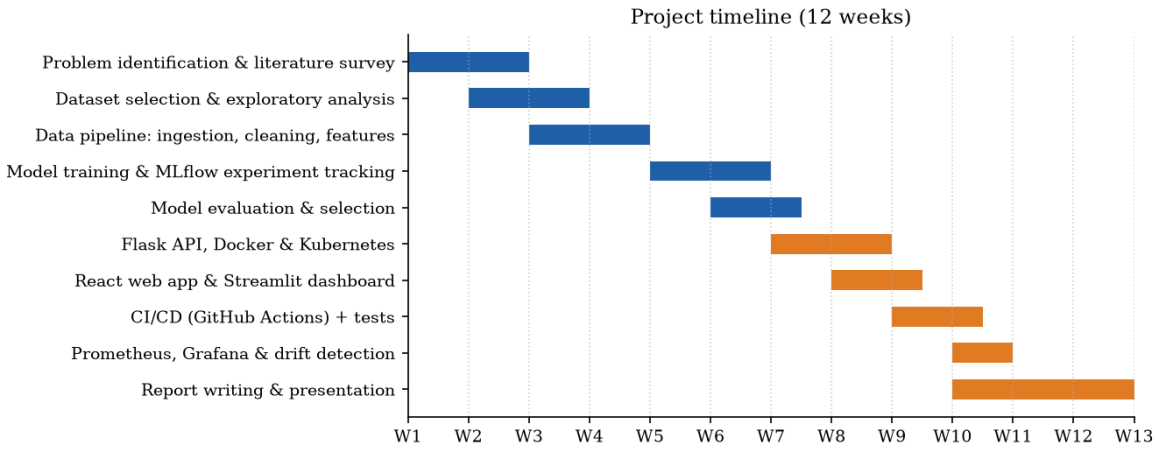


Figure 1.1*: Gantt chart of the project timeline*

1.5. Organization of the Report

- **Chapter 1** introduces the need, the problem, the tasks and the timeline.
- **Chapter 2** reviews the history of load forecasting and existing statistical, machine-learning and deep-learning solutions, compares them, and defines the problem and objectives mapped to the course outcomes.
- **Chapter 3** evaluates features and constraints (including ethical ones), compares two alternative designs, selects one and presents the implementation methodology.
- **Chapter 4** presents the implementation and validates the results: model comparison, tuning, classification metrics, deployment, CI/CD, monitoring and the responsible-AI analysis.
- **Chapter 5** concludes, discusses deviations from expectations and lists future work.
- References, Appendix and a step-by-step User Manual follow.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Table 2.1*: Timeline of the reported problem*

Period	Milestone
1970s	Box–Jenkins ARIMA methodology becomes the standard for utility-level load forecasting [6].
1992	Hart introduces Non-Intrusive Load Monitoring, showing household appliance loads can be inferred from aggregate meter data [7].
2000s	Machine-learning regressors (SVR, Random Forest [8], Gradient Boosting [9]) applied to building energy prediction.
2012	UCI publishes the Individual Household Electric Power Consumption dataset, which becomes a benchmark for household-level forecasting [10].
2015	Sculley et al. document "hidden technical debt" in production ML systems [4].
2016–2019	Deep learning (LSTM, CNN-LSTM) applied to household load forecasting [11], [12]; reviews of data-driven building energy prediction published [13].
2018	MLflow released to manage the ML lifecycle (tracking, projects, registry) [14].
2016–present	Probabilistic forecasting becomes the norm for load forecasting [15]; national smart-meter roll-outs (e.g. India's RDSS) make household data abundant [3].
2023	MLOps formally defined and its architecture surveyed in IEEE Access [5]; India enacts the Digital Personal Data Protection Act, relevant to smart-meter data [25].

2.2. Existing solutions

Statistical time-series models. ARIMA and exponential smoothing [6], [16] model linear autocorrelation and seasonality. They are interpretable and cheap but assume stationarity and struggle with the abrupt, appliance-driven spikes of a single household.

Classical machine learning. The standard supervised-learning toolkit, namely linear regression, decision trees, k-nearest neighbours, support vector machines and ensembles [21], [22], has been applied to building energy. Fan et al. combined data-mining techniques to predict next-day building consumption and peak demand [17]. Tree ensembles such as random forest [8] and gradient boosting [9], [18] learn non-linear interactions from engineered lag and calendar features and are robust to scale and outliers.

Deep learning. Marino et al. applied sequence-to-sequence LSTMs to the same UCI dataset [11]; Kim and Cho combined CNN and LSTM layers and reported lower error than

linear and LSTM baselines [12]. These models capture long dependencies but require more data, GPU training and careful tuning.

MLOps platforms. Kreuzberger et al. describe the components of a production ML system: versioned data and code, experiment tracking, model registry, automated pipelines, serving and monitoring [5]. Ameisen frames the same lifecycle from a product perspective, from prototype to monitored deployment [23]. MLflow [14], DVC and drift-detection methods [19] are widely used building blocks.

2.3. Bibliometric analysis

Table 2.2*: Bibliometric comparison of existing solutions*

Work	Approach	Key features	Effectiveness	Drawbacks
Box & Jenkins [6]	ARIMA	Linear autoregression, differencing	Good for aggregate, smooth loads	Poor on volatile single-house data; manual order selection
Fan et al. [17]	Ensemble data mining	Feature selection + multiple regressors	Accurate next-day building forecasts	Commercial buildings; no deployment
Marino et al. [11]	LSTM seq2seq	Learns temporal patterns directly	Competitive on UCI data at hourly scale	Overfits at minute scale; heavy training
Kim & Cho [12]	CNN-LSTM	Convolutional feature extraction + LSTM	Low error on UCI dataset	Black box; GPU cost; no monitoring
Amasyali & El-Gohary [13]	Review	Survey of 60+ studies	Shows ML methods outperform statistical ones in most cases	Notes lack of deployed, maintained systems
Kreuzberger et al. [5]	MLOps architecture	CI/CD/CT, registry, monitoring	Reference architecture	Not specific to energy forecasting

2.4. Review Summary

The literature shows that (i) machine-learning models with good lag and calendar features consistently beat purely statistical models on household data; (ii) deep networks can be marginally more accurate but at much higher training and operational cost; and (iii) almost all published household forecasting work stops at an offline accuracy number, with no serving, automation, monitoring or ethical review. This project therefore compares the standard algorithms under one protocol, tunes them, and wraps the best one in a complete MLOps pipeline, evaluated honestly against naive baselines on a chronological hold-out.

2.5. Problem Definition

What is to be done: build a system that, given the most recent week of hourly smart-meter readings for a household, forecasts the household's mean active power (kW) for each of the next 1–48 hours with a prediction interval, flags likely peak hours, and can be retrained, tested, deployed and monitored automatically.

How it is to be done: using the UCI household dataset, a reproducible DVC pipeline, supervised regression and classification models compared and tuned under MLflow tracking, a Flask REST API packaged with Docker and deployed to the cloud, a GitHub Actions CI/CD workflow, and Prometheus/Grafana and PSI-based monitoring.

What is not to be done: appliance-level disaggregation (NILM), control of household devices, billing integration, or collection of new personal data. Forecasts are for a single household and are not intended for grid-level dispatch or decisions about individuals.

2.6. Goals/Objectives

1. Clean and resample the 2.07 M-row dataset into a gap-free hourly series with an auditable data report, including missing-value and outlier analysis.
2. Engineer at least 25 features that use only past information (verified by an automated leakage test).
3. Train, tune (GridSearchCV) and compare the course algorithms, namely linear regression, decision tree, KNN, SVM and random forest, under experiment tracking on a strict chronological split.
4. Beat the persistence baseline on the held-out test period by at least 15% in MAE.
5. Build a peak-hour classifier and report accuracy, precision, recall, F1, the confusion matrix and ROC.
6. Serve predictions through a Flask REST API with input validation and response time under 1 s, containerised with Docker and deployed to a cloud platform.
7. Automate pipeline, quality gates, tests and container build with CI/CD; version data and models with DVC.
8. Monitor the service with Prometheus and Grafana, log predictions, and detect drift (PSI > 0.2).
9. Analyse fairness, transparency, privacy and security of the system.

Table 2.3 maps these objectives to the course outcomes of MLOps-1 (25CSH-320).

Table 2.3*: Mapping of project objectives to course outcomes*

Course outcome	How the project addresses it	Section
CO1: ML fundamentals, types, applications	Supervised regression and classification applied to a real energy-sector problem; algorithm survey	1.1, 2.2
CO2: Preprocessing, feature engineering, data handling	Missing values, IQR outliers, resampling, scaling, 31 engineered features, chronological split	3.1, 3.3, 4.1
CO3: Develop and evaluate models with various algorithms	9 regressors and 3 classifiers, GridSearchCV with time-series CV, over/under-fitting analysis, MSE/MAE/R^2 and accuracy/precision/recall/F1/ROC	3.6, 4.1
CO4: Deploy with APIs, microservices and cloud	Flask REST API, Docker Compose microservices, Kubernetes manifests, Vercel cloud deployment	4.1
CO5: CI/CD and ethical considerations	GitHub Actions + DVC + Vercel CD; fairness, transparency, privacy and security analysis; model card	3.2, 4.1

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Dataset. The UCI dataset [10] contains 2,075,259 one-minute measurements from 2006-12-16 to 2010-11-26 for one house in Sceaux, France: global active and reactive power, voltage, current intensity and three sub-meters (kitchen; laundry room; electric water heater and air conditioner). It was chosen because it is public, long enough to cover four winters, contains real gaps, and mirrors the load-profile channels exposed by IEC 62056 smart meters. It is stored as a single CSV file versioned with DVC. Figures 3.1–3.3 summarise the exploratory analysis.

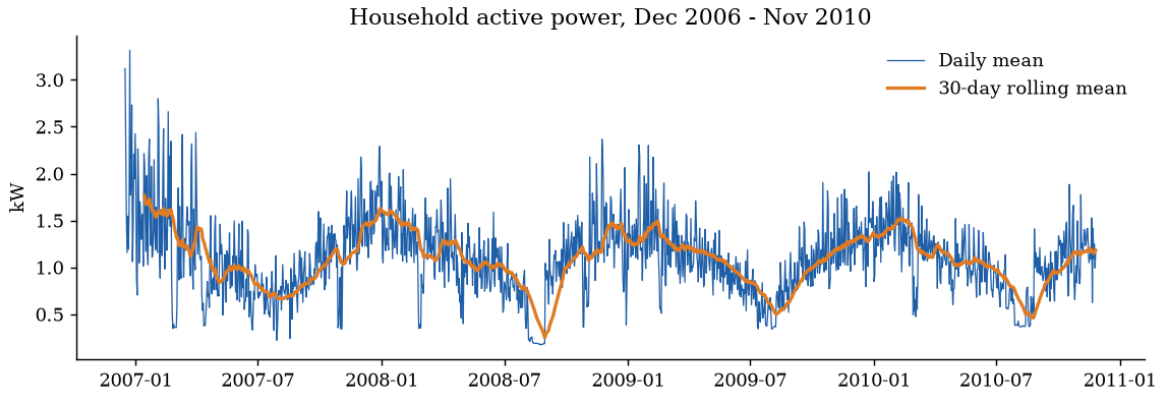


Figure 3.1*: Daily mean active power with 30-day rolling mean (strong yearly seasonality)*

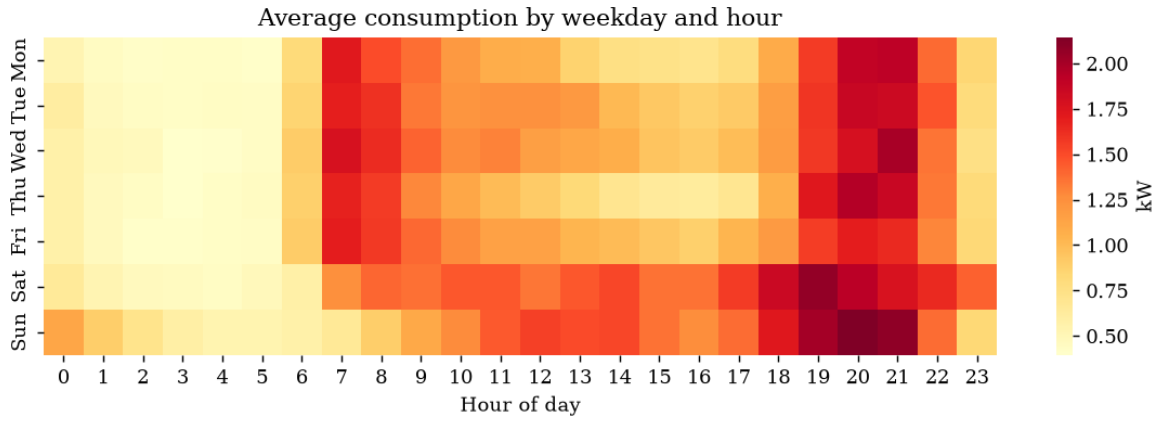


Figure 3.2*: Average consumption by weekday and hour (morning and evening peaks)*

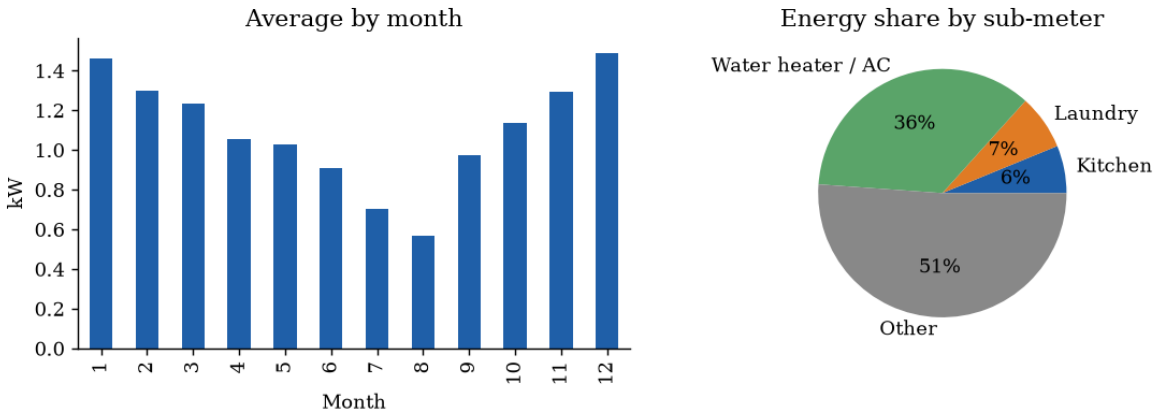


Figure 3.3: *Monthly pattern and share of energy per sub-meter*

The EDA shows a strong winter peak (consumption roughly doubles from August to December), two daily peaks around 07:00 and 20:00–21:00, higher daytime use at weekends, and an August holiday dip.

Outliers. With the inter-quartile-range rule (outside Q1 − 1.5·IQR … Q3 + 1.5·IQR), 738 hourly values of active power (2.13%) lie above 3.431 kW (Figure 3.4). These are genuine peak-usage hours (heaters, ovens), exactly the hours the forecaster must learn, so they are kept rather than removed. Instead, distance- and margin-based models (KNN, SVM) receive standardised inputs so that large-valued columns do not dominate.

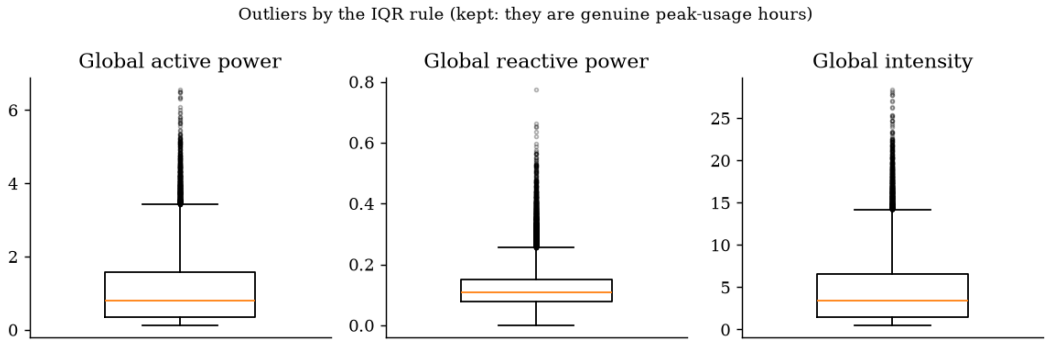


Figure 3.4: *Box plots of hourly readings; points beyond the whiskers are IQR outliers*

These findings drove the feature specification in Table 3.1.

Table 3.1: *Selected feature specification (31 features)*

Feature group	Features	Rationale
Autoregressive lags	lag 1, 2, 3, 6, 12, 24, 48, 168 h	Short-term persistence, same hour yesterday and last week
Rolling statistics	mean and std over 3, 6, 24, 168 h	Recent level and volatility
Calendar	hour, weekday, month, weekend, French public holiday	Routines and seasonality
Cyclical encodings	sin/cos of hour and month	23:00 and 00:00 are neighbours
Electrical context	reactive power, voltage, intensity,	Which appliances were active

(t−1)	3 sub-meters	

3.2. Design Constraints

Table 3.2*: Design constraints*

Constraint	Consideration in the design
Standards	ML life-cycle stages follow ISO/IEC 23053; software quality attributes of ISO/IEC 25010 (reliability, maintainability) drive testing and modularity; metrics exposed in the OpenMetrics format.
Economic / Cost	Must run on a student laptop, free CI runners and a free cloud tier: no GPU, only open-source tools (scikit-learn, MLflow, DVC, Flask, Docker, Prometheus, Grafana).
Environmental	Low-compute models reduce the energy footprint of retraining; the purpose of the system is itself to support energy saving (ISO 50001 spirit).
Health & Safety	Advisory system only: it never switches appliances, so incorrect forecasts cannot cause physical harm.
Professional / Ethical	Honest evaluation against naive baselines on unseen future data; no data leakage; limitations and error differences between groups of hours are disclosed (model card).
Fairness & bias	The data comes from one French household (2006–2010); quality of service is measured across day type, time of day, season and usage level; the model is not claimed to generalise to other homes or climates without re-validation.
Privacy	Fine-grained meter data can reveal occupancy and routines. Only a public anonymised dataset is used; the prediction log stores forecasts, never submitted readings; deployment on real data must follow India's Digital Personal Data Protection Act, 2023 [25].
Security	All API input is validated (types, ranges, minimum history, duplicates); request size is limited; only models produced by this pipeline are loaded; rejected requests are monitored.
Manufacturability / Deployability	Must be reproducible on any machine: pinned configuration, DVC pipeline and lock file, Docker image, single-command reproduction.
Regulatory	Indian smart meters follow IS 16444; the API input schema maps to the hourly load profile such meters provide.

3.3. Analysis of Features and finalization subject to constraints

- **Removed:** minute-level forecasting. At one-minute resolution the series is dominated by appliance switching noise; hourly resolution matches billing and demand-response use and reduces data 60×.
- **Removed:** weather features. They would improve accuracy but no free, historical, hourly weather feed for Sceaux 2006–2010 could be integrated within the cost constraint.
- **Removed:** deep LSTM model as the production candidate, due to the no-GPU and explainability constraints (kept as future work).

- **Modified:** missing data handling. Gaps up to 6 h are time-interpolated; longer outages are filled with the same hour of the previous week to preserve the daily profile.
- **Modified:** outliers are reported and kept (they are real peaks); feature scaling is applied only where the algorithm needs it (linear models, KNN, SVM).
- **Added:** a peak-hour classifier, because demand response needs a yes/no alert as well as a number.
- **Added:** prediction intervals ($\pm 1.96\sigma_r\sqrt{h}$) so users can see forecast uncertainty.
- **Added:** automated leakage test, quality gates in CI, and a bias analysis across groups of hours to satisfy the professional/ethical constraints.

3.4. Design Flow

Two alternative designs were developed to meet the specification (Figure 3.5).

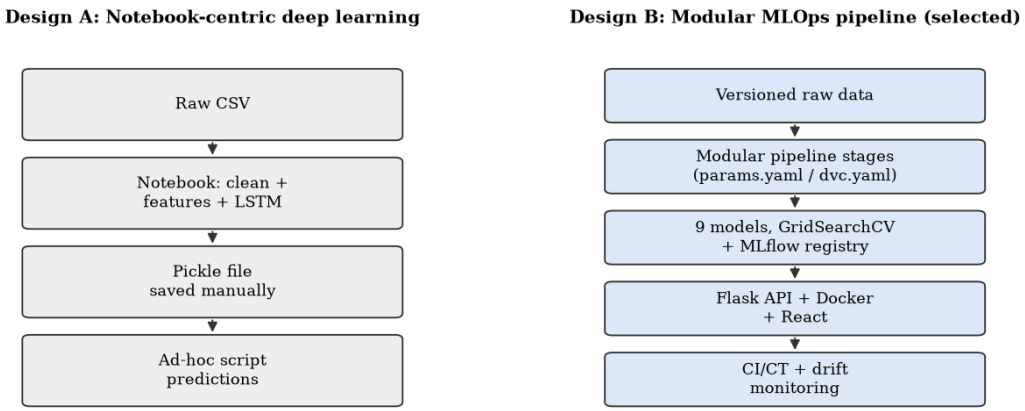


Figure 3.5*: Alternative designs considered*

Design A – Notebook-centric deep learning. All steps (cleaning, features, an LSTM model) live in a single Jupyter notebook. The trained network is pickled manually and predictions are produced with an ad-hoc script. This is the typical approach in the literature [11], [12].

Design B – Modular MLOps pipeline. Each stage is an independent, tested Python module driven by one configuration file (params.yaml) and declared as a DVC pipeline. All course algorithms are compared and tuned under MLflow tracking; the best is registered and served by a Flask REST API in Docker, deployed to the cloud behind a React web app; GitHub Actions automates the pipeline, and Prometheus, Grafana and a drift monitor close the loop.

3.5. Design selection

Table 3.3*: Comparison of alternative designs*

Criterion	Design A	Design B
Expected accuracy	High (with tuning)	High (gradient boosting)
Training cost / hardware	GPU preferred, minutes–hours	CPU, about 4 min including tuning
Reproducibility	Low: hidden notebook state	High: params.yaml + DVC + Docker
Experiment tracking	Manual	Automatic (MLflow)
Deployment	Manual script	Flask API + container + cloud + web app
Testing / CI	None	11 automated tests, CI quality gates
Monitoring & retraining	None	Prometheus, Grafana, PSI drift, weekly CT
Explainability & ethics	Low	Feature importance, bias analysis, model card
Fit with constraints	Violates cost & reproducibility	Satisfies all

Design B is selected. It meets every constraint in Section 3.2, its accuracy is competitive, and it delivers the operational capabilities (automation, deployment, monitoring, responsible AI) that are the core outcomes of the course and that the literature identifies as missing.

3.6. Implementation plan/methodology

Figure 3.6 shows the algorithm executed by the pipeline and Figure 3.7 the detailed block diagram.

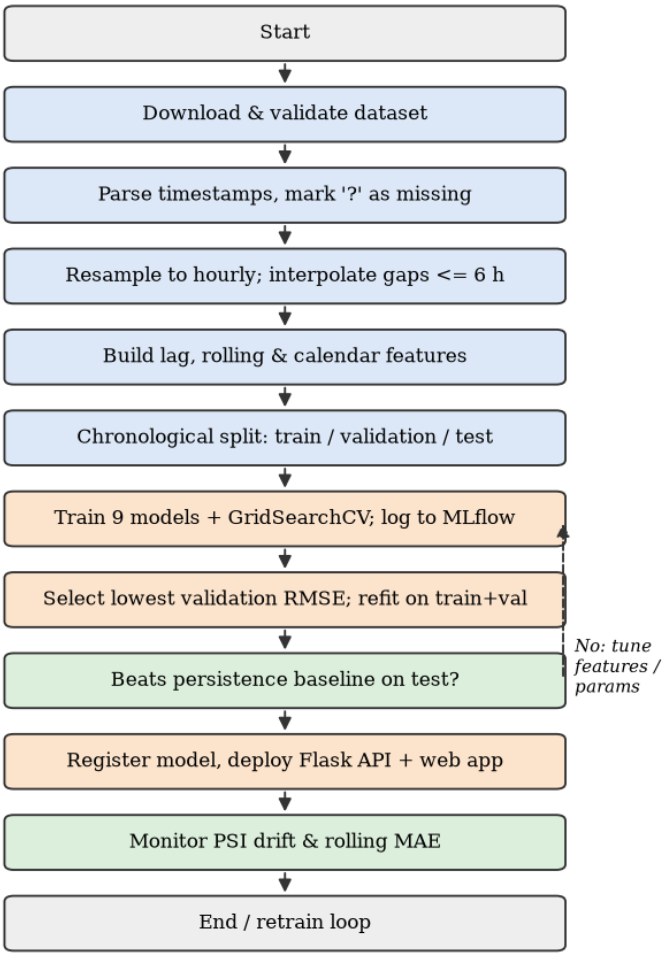


Figure 3.6*: Flowchart of the training and deployment algorithm*

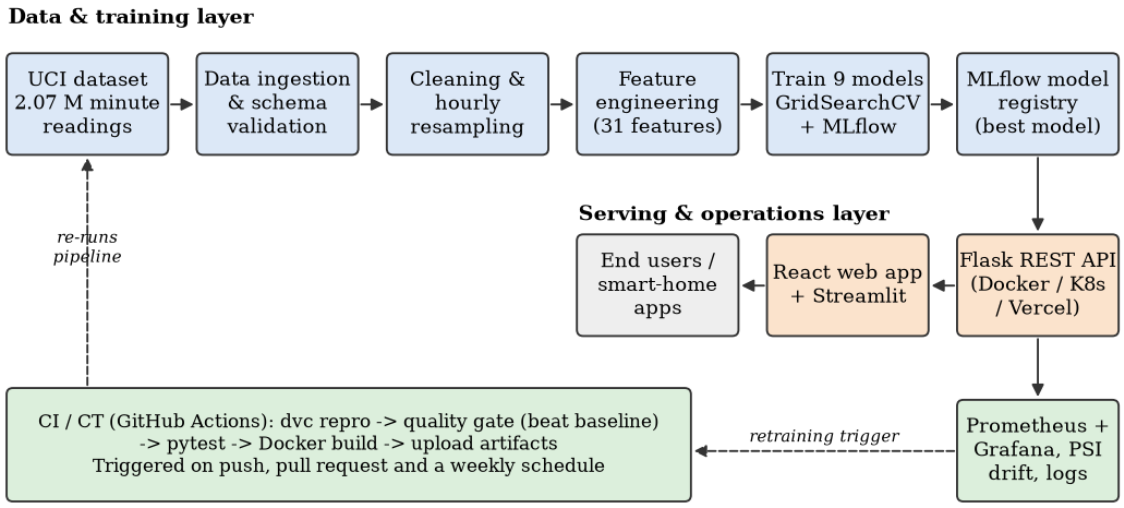


Figure 3.7*: Detailed block diagram of the MLOps architecture*

Data split. To respect time ordering, data is split chronologically: training up to 31 Dec 2009 (26,503 h), validation Jan–May 2010 (3,624 h) and test 2010-06-01 to 2010-11-26

(4,294 h). Models are selected on validation, refit on train+validation, and scored once on test. A random split is deliberately avoided: it would leak future information into training and overstate accuracy.

Regression models. Persistence ($\hat{y}_t = y_{t-1}$) and seasonal-naive ($\hat{y}_t = y_{t-24}$) baselines; linear regression and ridge regression (standardised inputs); decision tree; k-nearest neighbours (k = 10, standardised); support vector regression (RBF kernel, C = 1, ε = 0.1, standardised); random forest (200 trees, depth 16) [8]; histogram gradient boosting (500 iterations, η = 0.05, 31 leaves) [9], [18]. All are implemented with scikit-learn [20].

Hyper-parameter tuning. GridSearchCV searches the grids in params.yaml using TimeSeriesSplit with five folds, so every fold trains on the past and validates on the following period, which is the time-series form of cross-validation [21]. Tuned and default variants are both compared on the validation window; regularisation is controlled through ridge α, tree depth and minimum leaf size.

Peak-hour classification. An hour is labelled a peak hour when consumption exceeds τ = 1.59 kW, the 75th percentile of training consumption. Logistic regression, a default decision tree and a GridSearchCV-tuned decision tree (F1 scoring, balanced class weights allowed) are compared on validation F1.

Metrics. Regression: $MAE = (1/n)\Sigma|y_t - \hat{y}_t|$, $MSE = (1/n)\Sigma(y_t - \hat{y}_t)^2$, $RMSE = \sqrt{MSE}$, R^2 and MAPE; RMSE selects the model because peak errors matter most for demand response. Classification: accuracy = (TP+TN)/n, precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = harmonic mean of precision and recall, plus the confusion matrix and ROC-AUC [21], [22].

Monitoring. $PSI = \Sigma(q_i - p_i)\cdot\ln(q_i/p_i)$ over 10 quantile bins of the training distribution; PSI > 0.2 signals significant drift. Retraining is recommended when any key feature drifts or recent 30-day MAE degrades more than 20%. At run time Prometheus scrapes request, latency, forecast and alert counters from the API every 15 s.

Table 3.4*: Software modules of the implementation*

Module	Responsibility
src/data_ingestion.py	Download the UCI archive, convert to CSV, verify schema
src/preprocess.py	Parse, fill gaps, resample hourly, IQR outlier report
src/features.py	Build 31 leakage-free features
src/train.py	9 regressors, GridSearchCV, MLflow tracking and model registry
src/classify.py	Peak-hour classifier and classification metrics

src/monitor.py	PSI drift and performance degradation, retraining decision
src/evaluate.py	EDA, diagnostics, validation curve, classification and bias figures
src/predict.py	Recursive multi-step forecasting with intervals and peak alerts
serving/main.py, app.py	Flask REST API, Pydantic validation, Prometheus metrics, prediction log
web/ (React), dashboard/app.py	Web application (Vercel) and Streamlit dashboard
monitoring/, docker-compose.yml, k8s/	Prometheus, Grafana, five-service stack, Kubernetes manifests
dvc.yaml, .github/workflows/ci.yml	Versioned pipeline and CI/CD automation
tests/test_pipeline.py	11 unit, leakage, classification, API and monitoring tests

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

Modern tools used. Analysis: pandas, NumPy, Matplotlib, Seaborn, Plotly. Design drawings: architecture, flow and Gantt diagrams generated in Matplotlib. Modelling: scikit-learn. Experiment management and versioning: MLflow (tracking and model registry), DVC, Git/GitHub. Deployment: Flask, gunicorn, Docker, Docker Compose, Kubernetes manifests, Vercel. Operations: GitHub Actions, Prometheus, Grafana. Testing and validation: pytest, PSI drift analysis. Report preparation: python-docx, generated directly from pipeline outputs, so every number in this chapter is reproducible with dvc repro.

4.1.1. Data preparation

1.252% of minute readings were missing ('?'). After cleaning and hourly resampling the series has 34,589 gap-free records with mean 1.09 kW (median 0.80 kW, max 6.56 kW). Feature building drops the first 168 h (needed for the weekly lag), leaving 34,421 modelling rows. Figure 4.1 shows that current intensity is almost perfectly correlated with active power, while voltage is weakly negatively correlated.

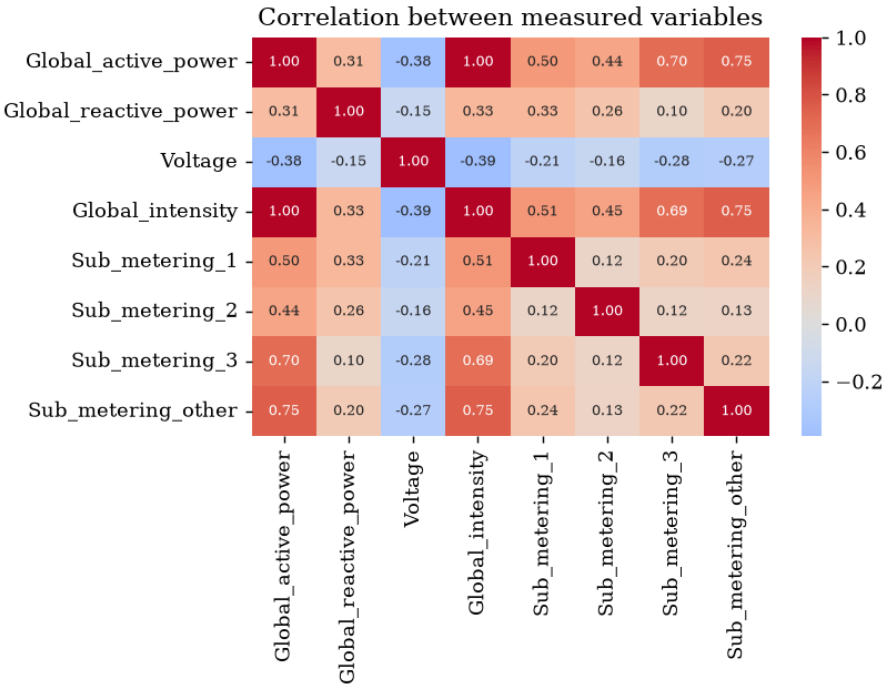


Figure 4.1*: Correlation matrix of the measured variables*

4.1.2. Model development and comparison

Table 4.1 compares all nine regression models with default hyper-parameters; every run is logged in MLflow (Figure 4.3 in this section shows the tracking UI).

Table 4.1*: Training and validation performance of all models (default hyper-parameters)*

Model	Train MAE	Val MAE	Val MSE	Val RMSE	Val R^2	Fit (s)
Persistence baseline	0.449	0.441	0.447	0.669	0.364	0.0
Seasonal naive (24 h)	0.663	0.636	0.832	0.912	-0.184	0.0
Linear regression	0.409	0.398	0.310	0.557	0.559	0.0
Ridge regression	0.409	0.398	0.310	0.557	0.559	0.0
Decision tree	0.000	0.506	0.602	0.776	0.142	0.6
K-nearest neighbours (KNN)	0.347	0.395	0.321	0.566	0.544	0.0
Support vector machine (SVR)	0.296	0.355	0.282	0.531	0.598	25.8
Random forest	0.220	0.345	0.259	0.509	0.632	4.7
Histogram gradient boosting	**0.289**	**0.341**	**0.248**	**0.498**	**0.647**	**0.8**

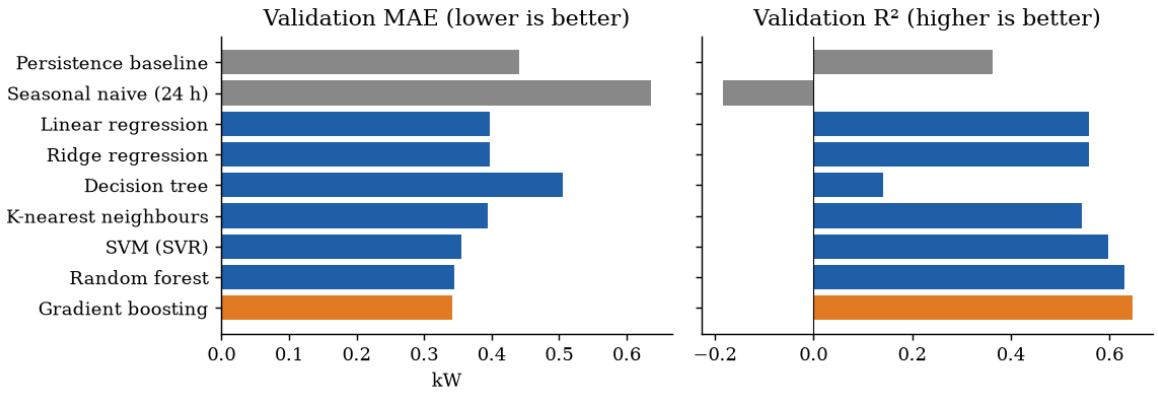


Figure 4.2*: Validation MAE and R^2 of all models (best in orange, baselines in grey)*

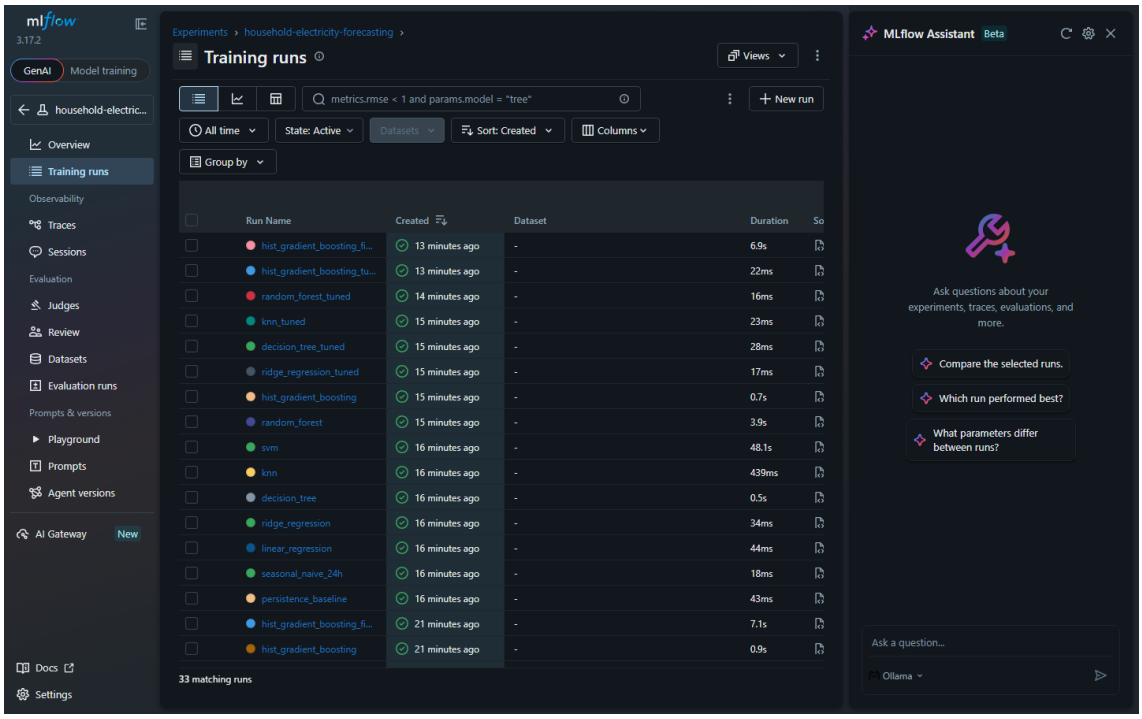


Figure 4.3: *MLflow tracking UI: runs of every model, tuned variant and final model*

Gradient boosting has the lowest validation RMSE (0.498 kW), closely followed by random forest and the SVM. Linear and ridge regression are identical at $\alpha = 1$ and form a strong linear reference (R^2 0.559). The seasonal-naive model has negative R^2: yesterday's same hour is a worse predictor than the mean, which confirms that household load is driven by irregular routines.

4.1.3. Over-fitting, under-fitting and regularisation

Figure 4.4 compares training and validation error. The default decision tree has a training MAE of 0.000 kW (it memorises every training hour) but the worst validation MAE of the machine-learning models (0.506 kW): a textbook case of over-fitting. Linear regression shows the opposite pattern, with training and validation errors almost equal, indicating mild under-fitting. The validation curve in Figure 4.5 shows both regimes for the tree: shallow trees under-fit, deep trees over-fit, and the best depth lies in between.

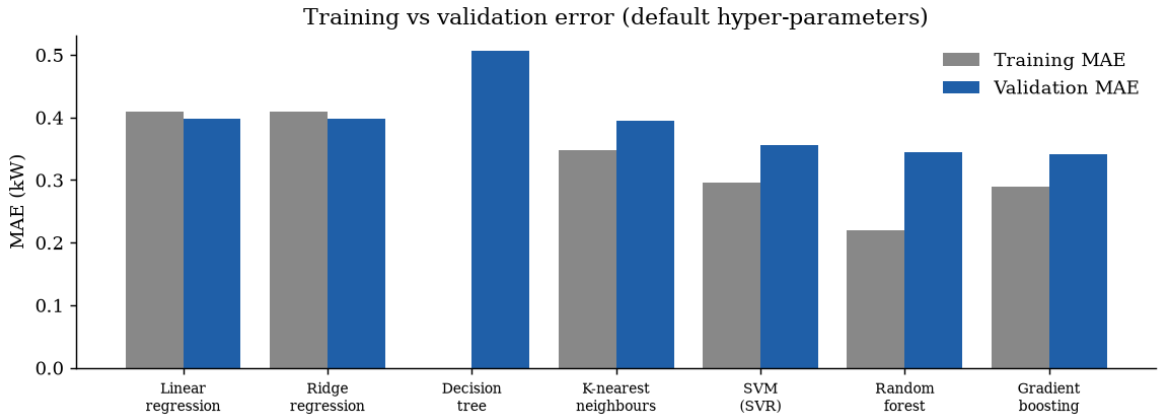


Figure 4.4: *Training vs validation MAE of each model (default hyper-parameters)*

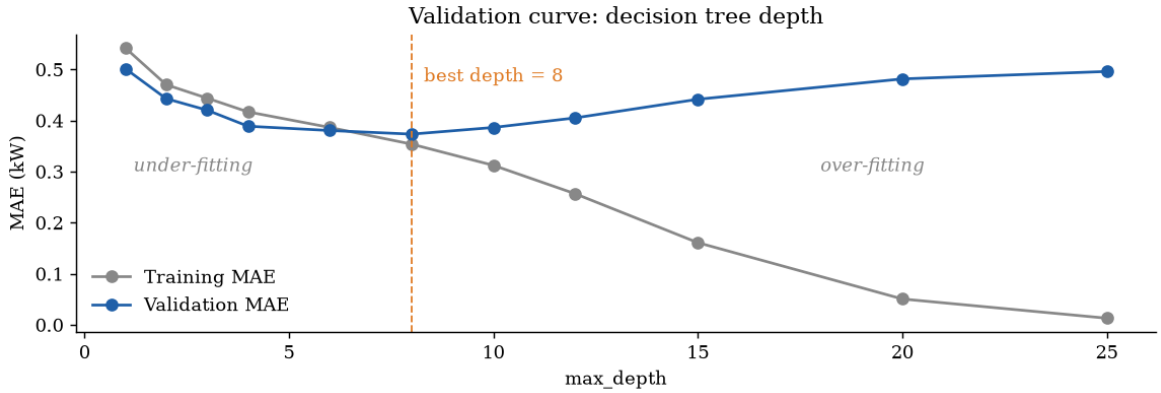


Figure 4.5: *Validation curve of the decision tree: error against max_depth*

4.1.4. Hyper-parameter tuning with GridSearchCV

Table 4.2 lists the GridSearchCV results (five time-series folds on the training period).

Table 4.2: *Hyper-parameter tuning results*

Model	Best parameters	Grid size	CV MAE	Val MAE default	Val MAE tuned
Ridge regression	alpha=100.0	5	0.403	0.398	0.398
Decision tree	max_depth=10, min_samples_leaf=100	24	0.397	0.506	0.380
K-nearest neighbours (KNN)	n_neighbors=40, weights=distance	8	0.406	0.395	0.388
Random forest	max_depth=16, min_samples_leaf=20	9	0.368	0.345	0.348
Histogram gradient boosting	learning_rate=0.03, max_leaf_nodes=15	9	0.363	0.341	0.344

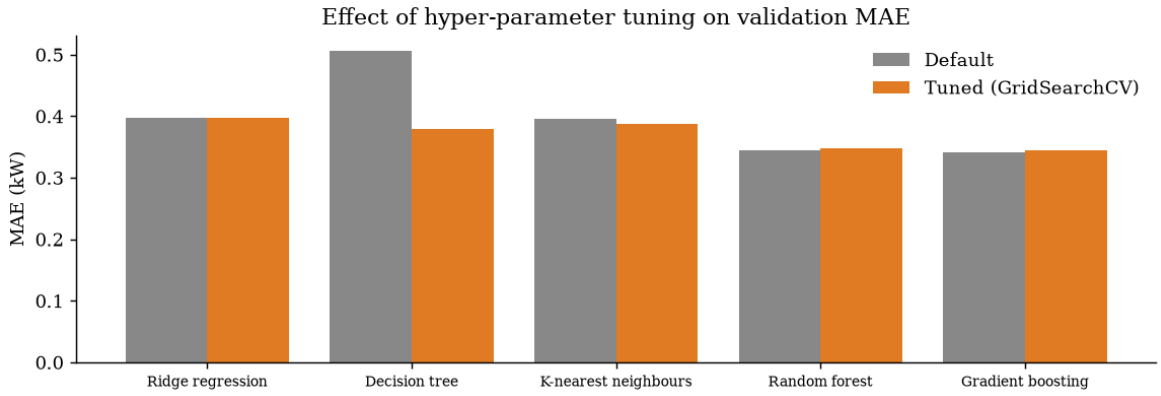


Figure 4.6*: Validation MAE before and after GridSearchCV tuning*

Tuning helps most where the default was poorly regularised: the decision tree improves from 0.506 to 0.380 kW once depth and minimum leaf size are limited. KNN gains by averaging over more neighbours with distance weighting; ridge prefers strong regularisation ($\alpha = 100.0$) but gains little. For random forest and gradient boosting the defaults were already well chosen, and the small differences are within the noise between cross-validation and the validation window. Default gradient boosting therefore remains the selected model.

4.1.5. Test-set results of the forecaster

The selected model was refit on train+validation and evaluated once on the untouched test period (Table 4.3).

Table 4.3*: Held-out test performance (Jun–Nov 2010)*

Model	MAE (kW)	MSE	RMSE (kW)	R^2	MAPE (%)
Persistence baseline	0.377	0.329	0.574	0.335	46.9
Histogram gradient boosting	**0.293**	**0.187**	**0.432**	**0.623**	**39.5**
Improvement	22.2%	43.3%	24.7%	+0.288	–

The forecasts follow the daily cycle and the timing of evening peaks closely (Figure 4.7). Residuals are centred near zero but right-skewed (Figure 4.8): the largest errors are sudden short spikes, such as an oven or water heater switching on, which are inherently unpredictable from past meter data alone.

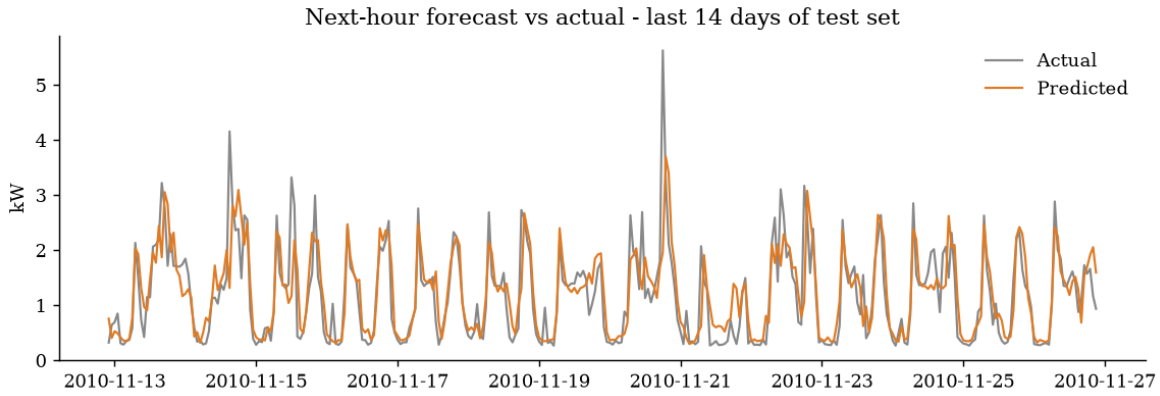


Figure 4.7*: Next-hour forecast vs actual consumption, last 14 days of the test set*

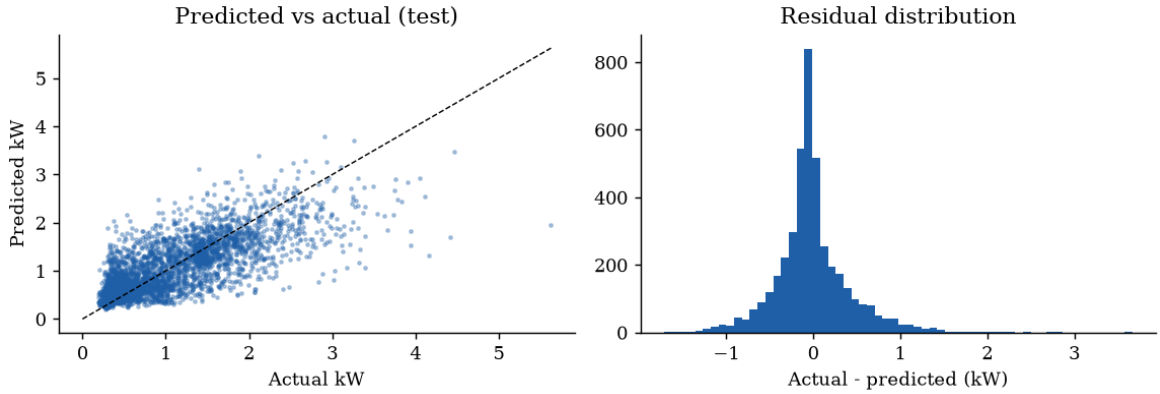


Figure 4.8*: Predicted vs actual scatter and residual distribution*

Explainability. Permutation importance (Figure 4.9) shows the inputs the model relies on: Global_intensity_lag1 (0.245), lag_1 (0.070), Sub_metering_3_lag1 (0.039), hour_cos (0.014). The previous hour's current and power carry the household's present state, the water-heater/AC sub-meter signals heavy thermal loads, and the hour encodings capture daily routines, which is physically consistent.

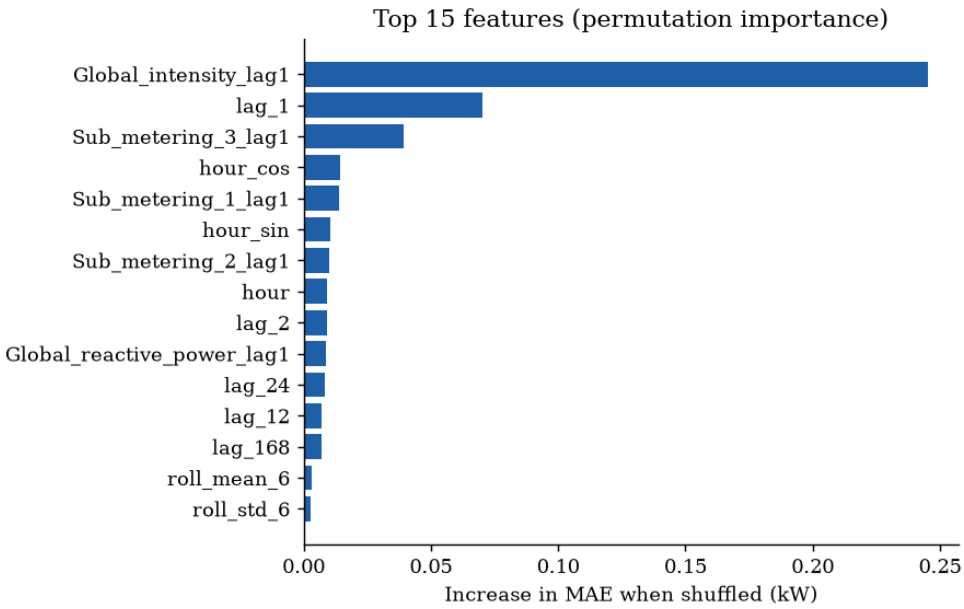


Figure 4.9*: Top 15 features by permutation importance*

4.1.6. Peak-hour classification

Peak hours make up 25% of training hours and 17.0% of the (summer-heavy) test period. Table 4.4 compares the classifiers on the validation window.

Table 4.4*: Peak-hour classifiers on the validation set*

Classifier	Train F1	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression	0.662	0.807	0.764	0.492	0.599	0.887
Decision tree default	1.000	0.770	0.610	0.589	0.599	0.717
Decision tree tuned	0.735	0.790	0.603	0.821	0.696	0.879

The default decision tree again over-fits (training F1 1.00). GridSearchCV selects max_depth = 12, min_samples_leaf = 50 and class_weight = balanced, raising validation F1 from 0.599 to 0.696, so the tuned tree is selected. On the test period it reaches accuracy 0.803, precision 0.454, recall 0.779, F1 0.574 and ROC-AUC 0.873 (Figure 4.10).

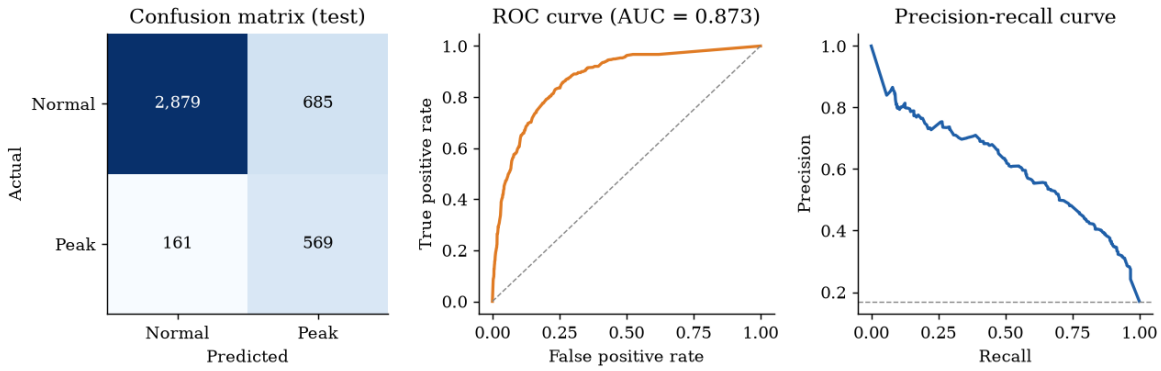


Figure 4.10*: Peak-hour classifier on the test set: confusion matrix, ROC and precision-recall curves*

Of 730 real peak hours, 569 were caught and 161 missed; 685 normal hours raised a false alarm. High recall with moderate precision is the safer trade-off for an alert, since a missed peak costs more than an unnecessary reminder. A simpler rule, flagging hours where the regression forecast exceeds τ, is more accurate (0.879) but catches only 60% of peaks. The deployed system uses the classifier for the next hour, and the forecast-threshold rule for later hours, where the classifier's inputs are no longer current. On 40 random test origins this rule was 77% accurate over hours 2–24, against 57% for the classifier.

4.1.7. Deployment

The forecaster and classifier are served by a Flask REST API (Table 4.5). Every request body is validated with Pydantic, which checks types, non-negative readings, positive voltage, a horizon of 1–48 h, at least 169 hours of history and no duplicate timestamps. Invalid input returns HTTP 422 with an explanation.

Table 4.5*: REST API endpoints*

Method	Endpoint	Purpose
GET	/api/health	Liveness and loaded model
GET	/api/model-info	Model, hyper-parameters, training window, test metrics, peak threshold
POST	/api/predict	Forecast + peak alerts from user-supplied hourly readings (JSON)
GET	/api/forecast	Forecast from a point in the test period, returned with actual values
GET	/api/sample-request	Ready-made request body for /api/predict
GET	/api/metrics	Prometheus metrics (OpenMetrics text format)

On-premise (Docker). A multi-stage Dockerfile builds the React web app with Node and then the Python runtime with gunicorn. Docker Compose starts five microservices: the API with the web app, the Streamlit dashboard, the MLflow UI, Prometheus and Grafana. The

full stack was built and run locally and every service responded. **Kubernetes.** k8s/deployment.yaml defines a two-replica Deployment with readiness and liveness probes on /api/health, a NodePort Service and a CPU-based HorizontalPodAutoscaler (2–5 pods). The manifests are validated in CI; they were not applied to a live cluster in this project. **Cloud.** The same Flask app runs as a serverless function on Vercel, with the React app served from its CDN, at https://voltcast-forecast.vercel.app. A 24-hour forecast request completes in well under one second.

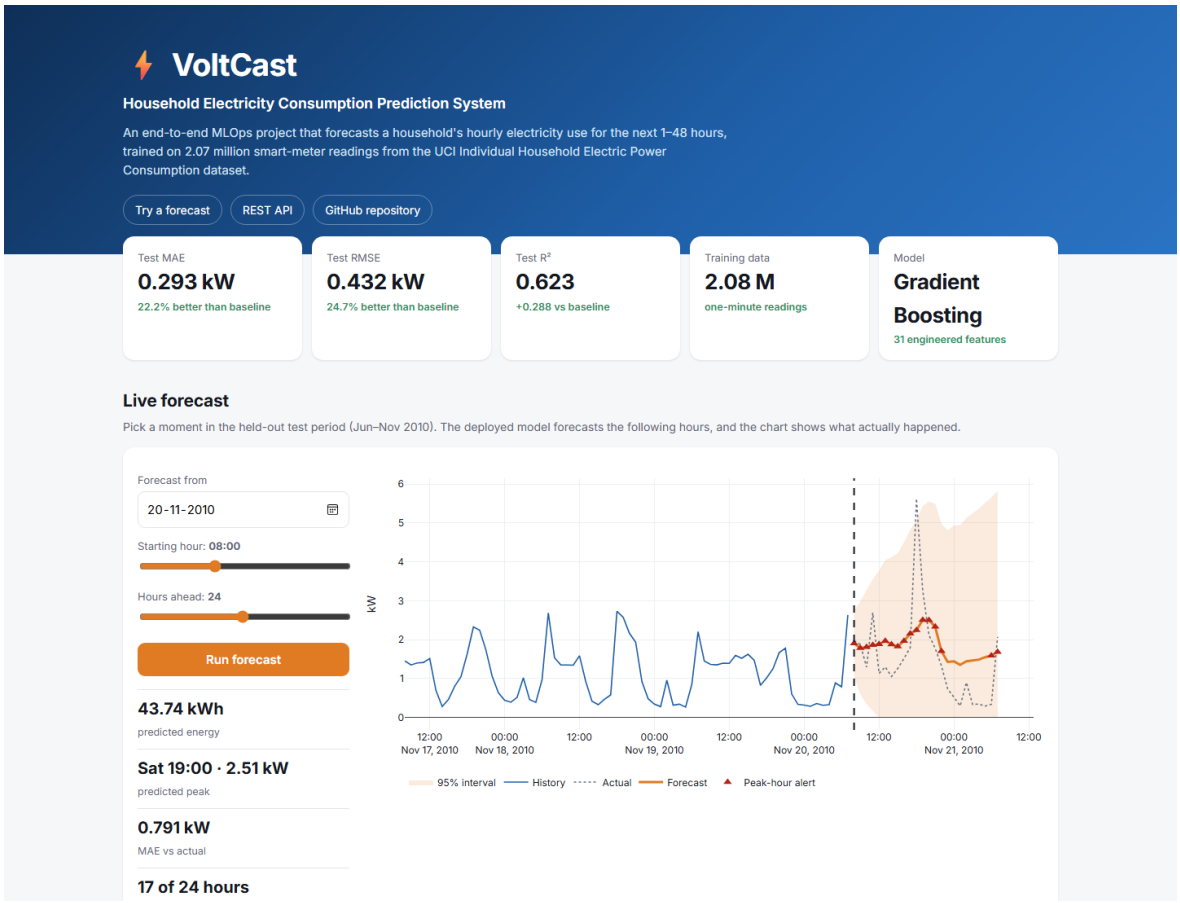


Figure 4.11*: VoltCast web application (React) with a live 24-hour forecast and peak-hour alerts*

4.1.8. CI/CD and versioning

The GitHub Actions workflow runs on every push, on pull requests and weekly (continuous training). It installs the dependencies, reproduces the pipeline with dvc repro, prints the DVC-tracked metrics, applies two quality gates (forecaster MAE below the persistence baseline; classifier ROC-AUC above 0.75), runs the 11 tests, builds the React app, validates the Docker Compose, Prometheus/Grafana and Kubernetes files, builds the Docker image and uploads the models and reports as artifacts. Vercel's Git integration

provides continuous deployment: every push to the main branch is built and published automatically.

Versioning. DVC records each stage's inputs, parameters and outputs in dvc.lock, so a given Git commit identifies the exact dataset, parameters, models and metrics; dvc repro re-runs only the stages whose inputs changed. The dataset is kept in the DVC cache rather than in Git. In addition, every trained forecaster is registered as a new version of voltcast-forecaster in the MLflow model registry.

Automated testing. Table 4.6 lists the pytest suite.

Table 4.6*: Automated test suite results*

Test	What it validates	Result
test_resample_fills_gaps	Missing hours are filled; output is gap-free	Pass
test_features_have_no_leakage	lag_1 and rolling means use only past values	Pass
test_metrics_perfect_prediction	Regression metric implementation	Pass
test_psi_detects_shift	PSI < 0.1 for same distribution, > 0.2 for shifted	Pass
test_classification_scores	Accuracy, precision, recall and confusion matrix	Pass
test_health_endpoint	Service liveness and model loading	Pass
test_predict_endpoint_returns_horizon	Horizon, interval ordering, peak alerts, strictly valid JSON	Pass
test_predict_rejects_short_history	Too little history returns HTTP 422	Pass
test_predict_rejects_invalid_reading	Negative voltage returns HTTP 422	Pass
test_prometheus_metrics_exposed	Prometheus counters and histograms are published	Pass
test_model_beats_baseline	Quality gate: model MAE < baseline MAE	Pass

4.1.9. Monitoring and logging

Prometheus scrapes /api/metrics every 15 seconds (Figure 4.12). The API exports request counts and latency histograms per endpoint, the number of forecast hours served, the distribution of forecast values, peak alerts, rejected requests and the loaded model's test MAE. A provisioned Grafana dashboard visualises them (Figure 4.13); invalid requests sent on purpose during the test run appear under 'Rejected requests'. Every prediction is also written as one JSON line to logs/predictions.jsonl (time, model, horizon, forecast values, number of peak hours), without the submitted meter readings.

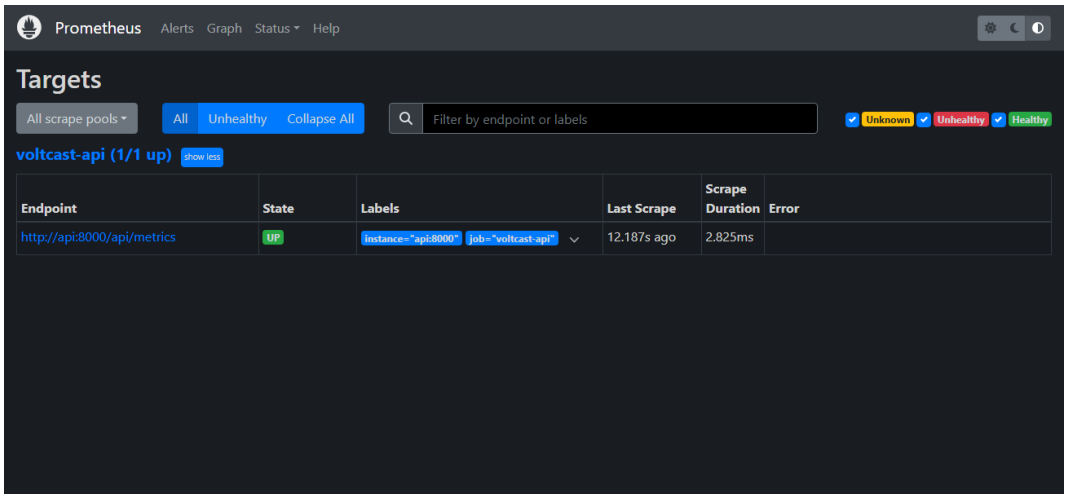


Figure 4.12*: Prometheus target page: the VoltCast API is being scraped (state UP)*

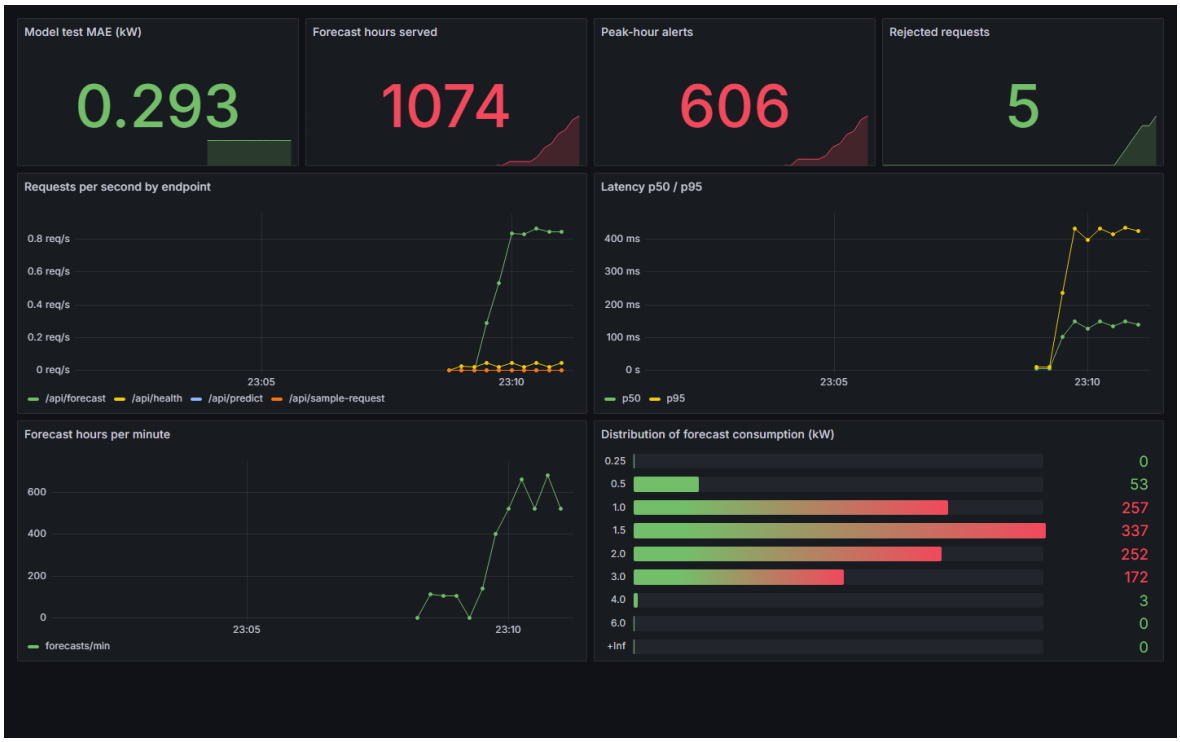


Figure 4.13*: Grafana dashboard "VoltCast – Model Monitoring" fed by Prometheus*

Data drift. Figure 4.14 shows the PSI between training and test distributions.

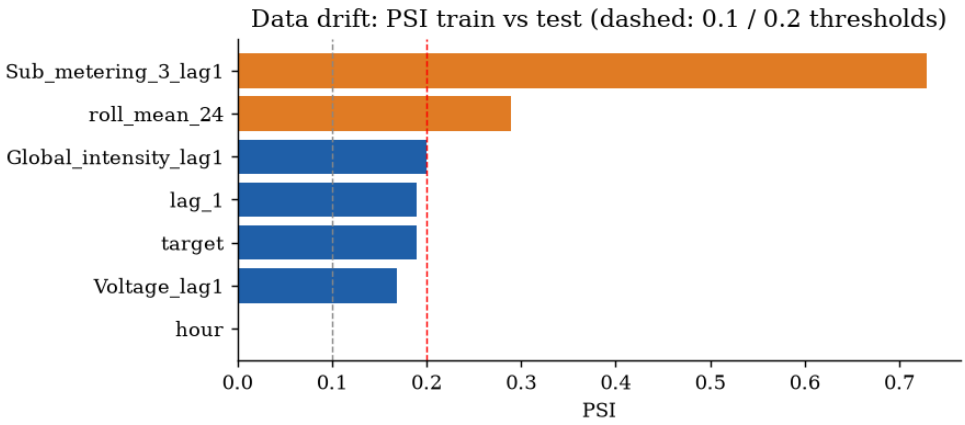


Figure 4.14*: Population Stability Index of monitored features (train vs test)*

Significant drift was detected in roll_mean_24, Sub_metering_3_lag1 (PSI 0.73 for the water-heater/AC sub-meter). This is genuine seasonal drift: the training data covers three winters, while the test period is June–November, when heating load is low. The 30-day rolling MAE (0.350 kW) was 19.2% above the reference test MAE as consumption climbed again in November. The monitor therefore set retraining_recommended = true, which is the behaviour a production MLOps loop requires.

4.1.10. Ethical considerations

Fairness and bias. The system makes no decisions about people, so fairness is assessed as equal quality of service across conditions [24]. Figure 4.15 breaks the test error down by group. Error is lowest at night (0.16 kW) and highest in the evening (0.39 kW) and in high-use hours (0.44 kW), and higher at weekends (0.33 vs 0.28 kW) and in autumn (0.33 vs 0.25 kW). The forecasts are therefore least reliable exactly when demand is highest, and this is stated to users. There is also sampling bias: one French household from 2006–2010 cannot represent Indian homes with air-conditioning loads, so the model must be re-validated before any wider use.

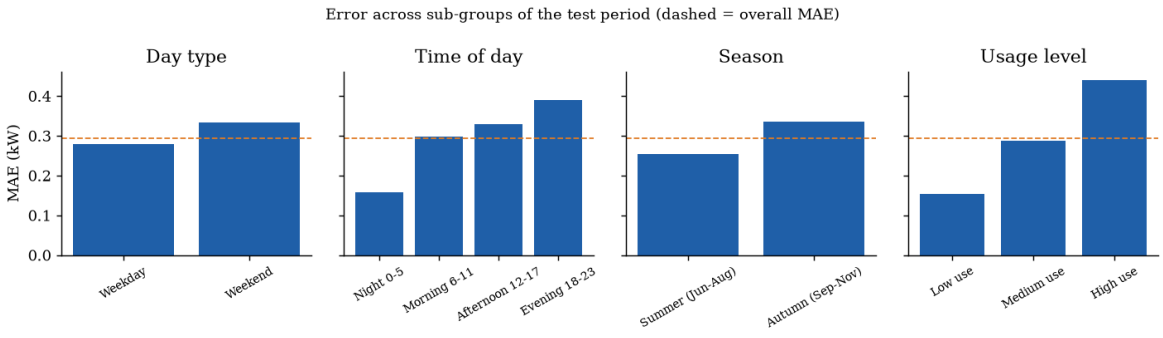


Figure 4.15*: Test error across sub-groups (dashed line: overall MAE)*

Transparency. Feature importance is published, every number is reproducible from the repository, forecasts carry a 95% interval, and a model card (MODEL_CARD.md) documents intended use, data, performance, limitations and monitoring. **Privacy.** Smart-meter data can reveal when residents are home or asleep. The project uses only a public anonymised dataset; the API does not store submitted readings; real deployments should aggregate data, obtain consent and comply with the Digital Personal Data Protection Act, 2023 [25]. **Security.** Input validation, a 2 MB request limit, loading only models produced by the pipeline, and monitoring of rejected requests protect the service against malformed or abusive input.



Figure 4.16*: Responsible-AI section of the web application*

4.1.11. Achievement of objectives

Table 4.7*: Achievement of objectives*

Objective (Section 2.6)	Target	Achieved
Clean hourly series	Gap-free, documented	34,589 hourly rows, 0 gaps, outlier report
Feature engineering	≥ 25 leakage-free	31 features, leakage test passes
Course algorithms compared and tuned	LR, DT, KNN, SVM, RF	9 regressors, 5 tuned with GridSearchCV
Improvement over baseline	≥ 15% MAE	22.2% MAE, 24.7% RMSE
Peak-hour classifier	Accuracy, P, R, F1, ROC	Accuracy 0.803, ROC-AUC 0.873
Flask API, Docker, cloud	< 1 s response	Flask + Docker + Vercel, sub-second
CI/CD and versioning	Pipeline + gates + tests + build	GitHub Actions, DVC, MLflow registry
Monitoring and logging	Prometheus, Grafana, drift	Dashboard live, 2 drifted features flagged

Ethics	Fairness, privacy, security	Bias analysis, model card, validation

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

VoltCast is a complete, working Household Electricity Consumption Prediction System built on MLOps principles. From 2.07 million raw smart-meter readings, a DVC-versioned pipeline produces a clean hourly series and 31 leakage-free features. Nine regression models, covering every algorithm in the course, are compared and tuned with GridSearchCV under MLflow tracking. The selected gradient boosting model forecasts with a test MAE of 0.293 kW and R^2 of 0.623, 22.2% better than the persistence baseline. A tuned decision tree flags peak hours with 78% recall and ROC-AUC 0.873. The models are served by a Flask API in Docker and on the Vercel cloud, protected by CI/CD quality gates and tests, observed through Prometheus and Grafana, and reviewed for fairness, privacy and security.

Expected vs obtained results. The ≥ 15% improvement over the baseline was met. However, the absolute R^2 (~0.62) is lower than the >0.9 values sometimes reported in the literature, for three reasons. (i) We forecast a single household at hourly resolution, where behaviour-driven spikes are inherently unpredictable. (ii) We use a strict chronological split and refuse features computed from the hour being predicted, whereas some studies shuffle data or use same-hour intensity, which inflates scores. (iii) The test period has a different seasonal distribution from training, which the monitor correctly flagged. MAPE (~39%) is inflated by very low night-time loads, where small absolute errors become large percentages; MAE, MSE and R^2 are the more meaningful measures here. Hyper-parameter tuning helped the poorly regularised models (decision tree, KNN) but not the ensembles, whose defaults were already near-optimal. The peak classifier's precision on the test period is lower than on validation because summer has fewer peak hours, which is another effect of seasonal drift.

5.2. Future work

- **Weather and occupancy inputs:** add outdoor temperature (heating/cooling degree hours) and holiday calendars from an API; expected to reduce winter peak error substantially.

- **Deep and probabilistic models:** evaluate LSTM/Temporal Fusion Transformer and quantile gradient boosting for calibrated prediction intervals [12], [15].
- **Multi-household, Indian data:** train a global model across many homes using data from IS 16444 smart meters, with federated learning to protect privacy.
- **Automated retraining:** connect the monitor's retraining flag to a workflow trigger and promote a new model only if it beats the current one (champion/challenger) in the MLflow registry.
- **Production Kubernetes:** apply the manifests to a managed cluster (AWS EKS, GCP GKE or Azure AKS) with Prometheus service discovery and Grafana alerting rules.
- **Actionable recommendations:** combine forecasts with time-of-day tariffs to suggest the cheapest time to run flexible appliances.

REFERENCES

[1] International Energy Agency, “Buildings – Energy System,” IEA, Paris. [Online]. Available: https://www.iea.org/energy-system/buildings

[2] Central Electricity Authority, “General Review (All India Electricity Statistics),” Ministry of Power, Government of India, New Delhi.

[3] Ministry of Power, Government of India, “Revamped Distribution Sector Scheme (RDSS),” 2021. [Online]. Available: https://recindia.nic.in/revamped-distribution-sector-scheme

[4] D. Sculley et al., “Hidden technical debt in machine learning systems,” in Advances in Neural Information Processing Systems 28, 2015.

[5] D. Kreuzberger, N. Kühl and S. Hirschl, “Machine Learning Operations (MLOps): Overview, definition, and architecture,” IEEE Access, vol. 11, pp. 31866–31879, 2023.

[6] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.

[7] G. W. Hart, “Nonintrusive appliance load monitoring,” Proceedings of the IEEE, vol. 80, no. 12, pp. 1870–1891, 1992.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] G. Hebrail and A. Berard, “Individual Household Electric Power Consumption,” UCI Machine Learning Repository, 2012. doi: 10.24432/C58K54.

[11] D. L. Marino, K. Amarasinghe and M. Manic, “Building energy load forecasting using deep neural networks,” in Proc. IECON 2016 – 42nd Annual Conference of the IEEE Industrial Electronics Society, 2016, pp. 7046–7051.

[12] T.-Y. Kim and S.-B. Cho, “Predicting residential energy consumption using CNN-LSTM neural networks,” Energy, vol. 182, pp. 72–81, 2019.

[13] K. Amasyali and N. M. El-Gohary, “A review of data-driven building energy consumption prediction studies,” Renewable and Sustainable Energy Reviews, vol. 81, pp. 1192–1205, 2018.

[14] M. Zaharia et al., “Accelerating the machine learning lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[15] T. Hong and S. Fan, “Probabilistic electric load forecasting: A tutorial review,” International Journal of Forecasting, vol. 32, no. 3, pp. 914–938, 2016.

[16] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne: OTexts, 2021.

[17] C. Fan, F. Xiao and S. Wang, “Development of prediction models for next-day building energy consumption and peak power demand using data mining techniques,” Applied Energy, vol. 127, pp. 1–10, 2014.

[18] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30, 2017.

[19] S. Rabanser, S. Günnemann and Z. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in Advances in Neural Information Processing Systems 32, 2019.

[20] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[21] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.

[22] A. C. Müller and S. Guido, Introduction to Machine Learning with Python: A Guide for Data Scientists. Sebastopol, CA: O'Reilly Media, 2016.

[23] E. Ameisen, Building Machine Learning Powered Applications: Going from Idea to Product. Sebastopol, CA: O'Reilly Media, 2020.

[24] A. Ng, Machine Learning Yearning, 2018. [Online]. Available: https://www.deeplearning.ai/machine-learning-yearning/

[25] Government of India, “The Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023, Ministry of Law and Justice, New Delhi, 2023.

APPENDIX

1. Plagiarism Report

2. Design Checklist

Table A.1*: Design checklist*

Item	Status
Problem identified and justified with documented sources	✓
Literature survey with comparison of existing solutions	✓
Measurable objectives defined and mapped to course outcomes	✓
Design constraints (standards, cost, ethics, privacy, security, safety) considered	✓
At least two alternative designs compared and one selected with reasons	✓
Flowchart / block diagram of implementation	✓
Data preprocessing: missing values, outliers, normalisation, chronological split	✓
Linear regression, decision tree, KNN, SVM and random forest implemented and compared	✓
GridSearchCV tuning, cross-validation, over-/under-fitting analysis	✓
Classification metrics: accuracy, precision, recall, F1, confusion matrix, ROC	✓
Version control with Git; data and pipeline versioning with DVC; MLflow registry	✓
Flask REST API, Docker containerisation, cloud deployment, Kubernetes manifests	✓
CI/CD with GitHub Actions and automatic deployment	✓
Monitoring with Prometheus and Grafana; prediction logging; drift detection	✓
Fairness, transparency, privacy and security analysed (model card)	✓
User manual with step-by-step instructions	✓

USER MANUAL

Hosted version (no installation): open https://voltcast-forecast.vercel.app for the web application and live forecasts. The API is at https://voltcast-forecast.vercel.app/api (for example /api/health). Source code: https://github.com/Kunal039/voltcast.

Prerequisites: Python 3.10 or newer, Node.js 20, about 1 GB free disk space, and internet access for the first run (dataset download, ~20 MB). Docker Desktop is needed for the monitoring stack.

Step 1: Get the project.

git clone https://github.com/Kunal039/voltcast.git then open a terminal inside the folder.

Step 2: Install dependencies.

pip install -r requirements-dev.txt

Step 3: Reproduce the pipeline.

dvc repro (downloads the dataset as CSV, preprocesses, builds features, trains and tunes all models, trains the peak classifier, runs monitoring and creates all figures; about 4 minutes). dvc metrics show prints the tracked metrics.

Step 4: Run the tests.

pytest -q (11 tests should pass)

Step 5: Inspect experiments in MLflow.

mlflow ui --backend-store-uri sqlite:///mlflow.db then open http://localhost:5000 to compare runs and see the registered model voltcast-forecaster.

Step 6: Start the web app and Flask API.

cd web && npm install && npm run build && cd .. then python app.py and open http://localhost:8000 (Figure M.1). Choose a date, starting hour and horizon and click Run forecast. The chart shows history, forecast, actual values, the 95% interval and peak-hour alerts.

Step 7: Call the API.

curl http://localhost:8000/api/sample-request > req.json then curl -X POST -H "Content-Type: application/json" -d @req.json http://localhost:8000/api/predict

Step 8: Run the full stack with monitoring.

docker compose up --build starts the API and web app (8000), Streamlit (8501), MLflow (5000), Prometheus (9090) and Grafana (3000). Open Grafana and select the dashboard "VoltCast – Model Monitoring" (Figures M.4–M.5).

Step 9: Deploy on Kubernetes (optional).

docker build -t voltcast:latest . then load the image into the cluster (e.g. minikube image load voltcast:latest) and run kubectl apply -f k8s/

Step 10: Retrain with new settings.

Edit configs/params.yaml (e.g. a hyper-parameter grid or split dates) and run dvc repro. Only affected stages re-run; a new MLflow run and model version are created.

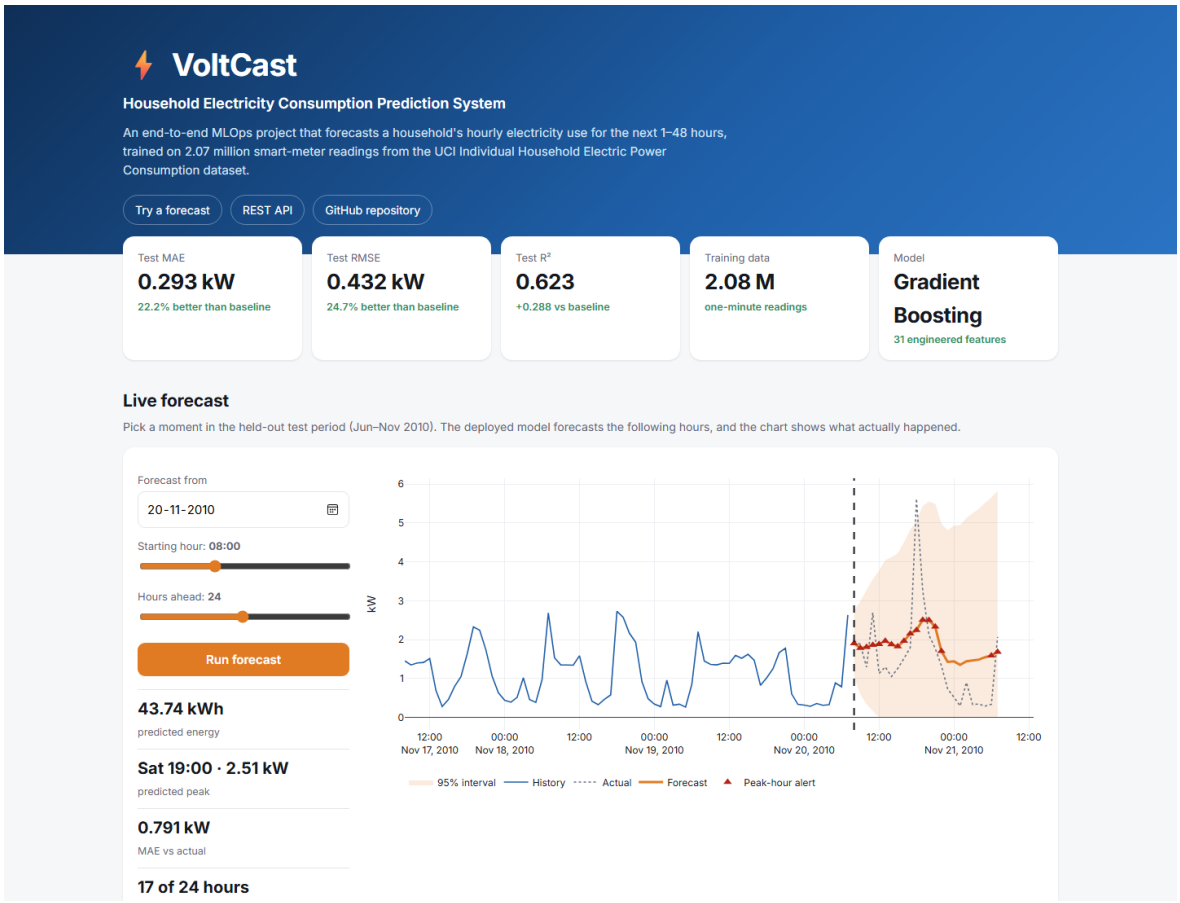


Figure M.1*: Web application – live forecast*



Figure M.2: *Web application – model performance, over-fitting and tuning*

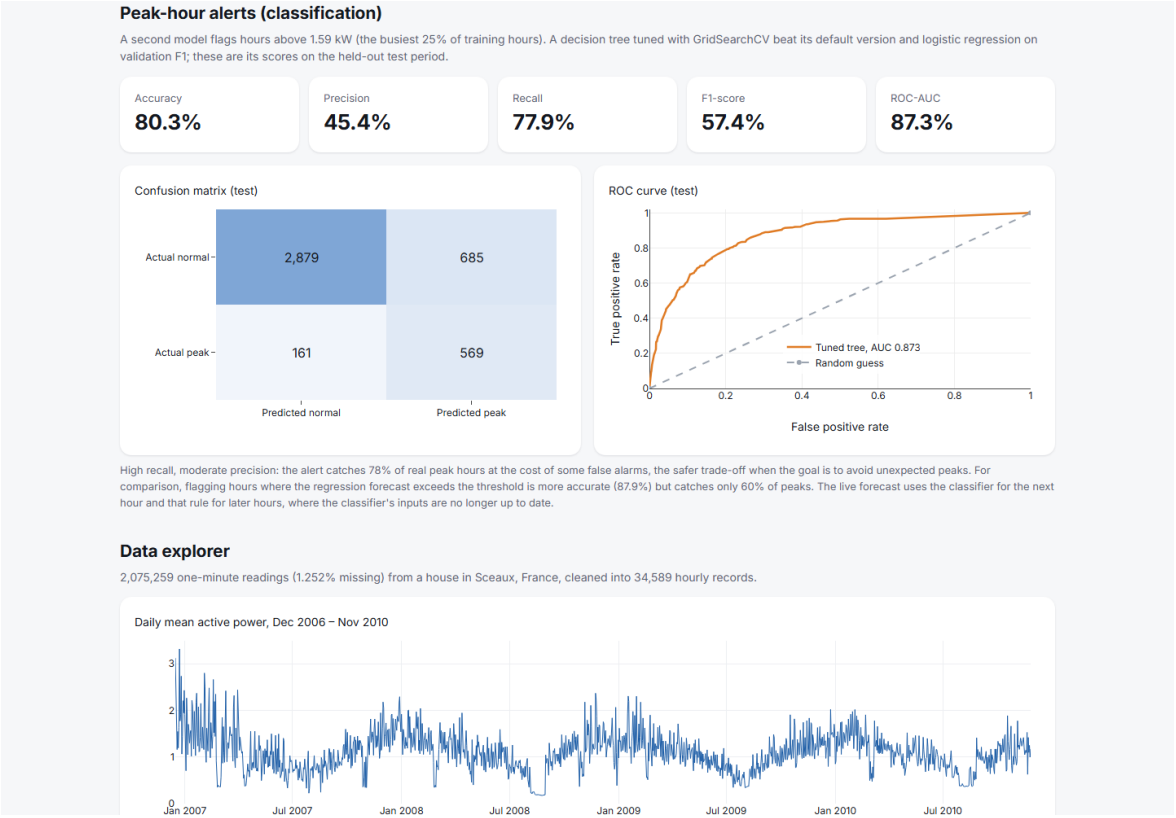


Figure M.3: *Web application – peak-hour classifier metrics*

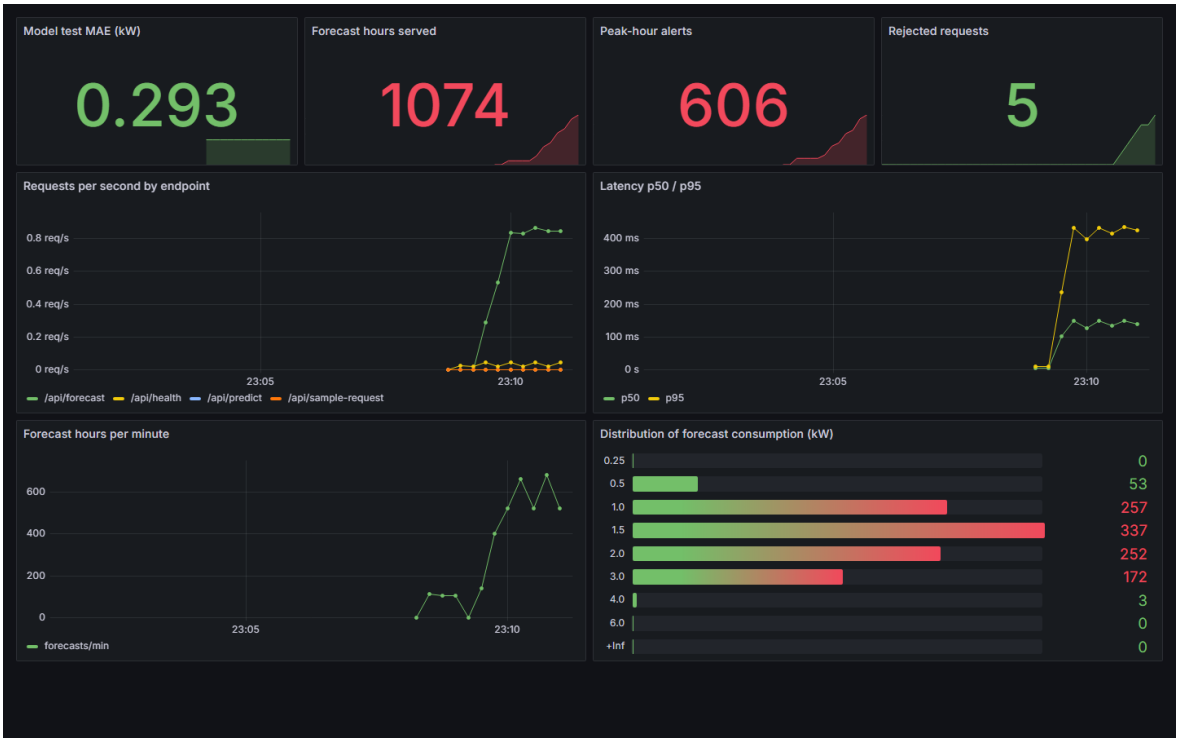


Figure M.4: *Grafana monitoring dashboard*

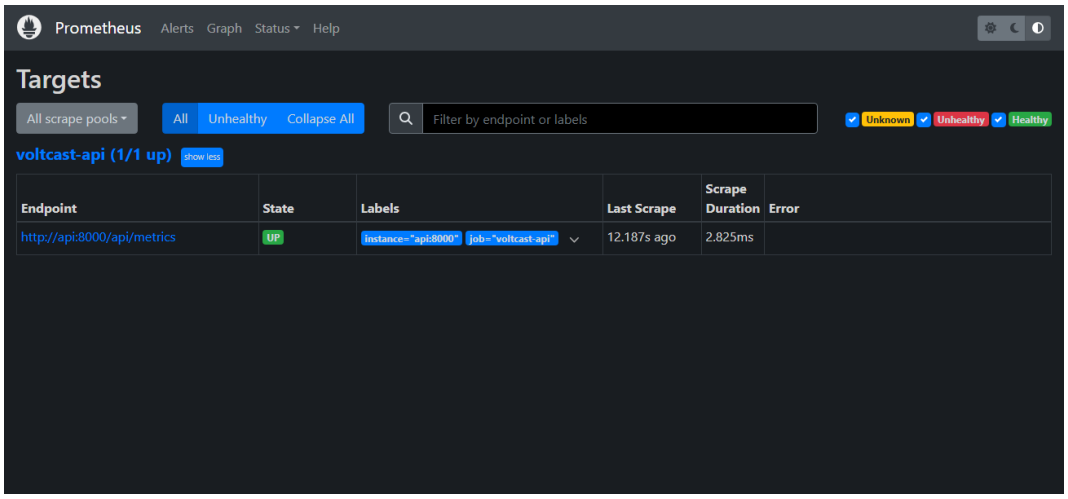


Figure M.5*: Prometheus targets page*

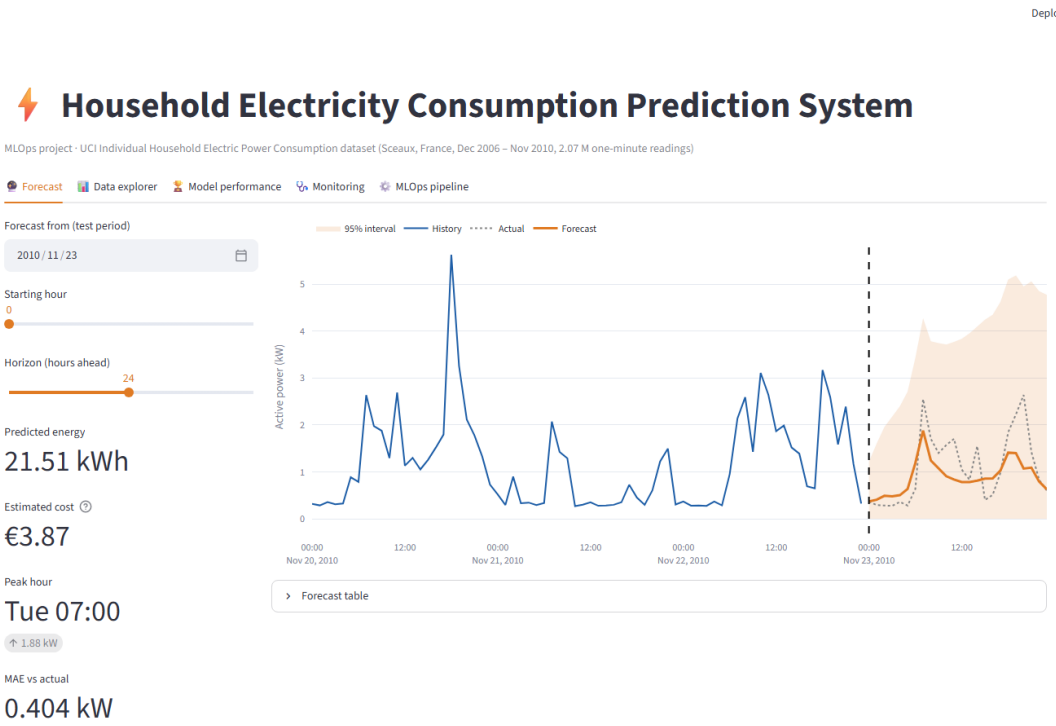


Figure M.6*: Streamlit dashboard (Docker / local) – forecast tab*